

PROGRAMMING FOR PROBLEM SOLVING

FUNDAMENTALS OF COMPUTER

1

PREVIOUS YEARS QUESTIONS

PART A

Q.1 What is an algorithm? Write an algorithm to print even numbers from 2 to 100. [R.T.U. 2019]

Ans. Algorithm : Algorithms are one of the most basic tools that are used to develop the problem-solving logic. An algorithm is defined as a finite sequence of explicit instructions that, when provided with a set of input values, produces an output and then terminates. In algorithm, after a finite number of steps, solution of the problem is achieved. Algorithms can have steps that repeat (iterate) or require decisions (logic and comparison) until the task is completed.

Different algorithms may accomplish the same task, with a different set of instructions, in more or less the same time, space, and efforts. For example, two different recipes for preparing tea, one "add the sugar" while "boiling the water" and the other "after boiling the water" produce the same result. However, performing an algorithm correctly does not guarantee a solution, if the algorithm is flawed or not appropriate to the context. For example, preparing the tea algorithm will fail if there were no tea leaves present, even if all the motions of preparing the tea were performed as if the tea leaves were there. We use algorithms in our daily life. For example, to determine the largest number out of three numbers A, B, and C, the following algorithm may be used.

Step 1: Start
Step 2: Read three numbers say A, B, C
Step 3: Find the larger number between A and B and store it in MAX_AB.

Step 4: Find the larger number between MAX_AB and C and store it in MAX.

Step 5: Display MAX

Step 6: Stop

The algorithm to print even number from 2 to 100 is as follows.

Step 1 : Start

Step 2 : Initialize NUM $\leftarrow$ 2

Step 3 : Execute Steps 4 and 5 Till NUM Equal 100

Step 4 : Calculate NUM $\leftarrow$ NUM + 2

Step 5 : Print NUM

Step 6 : Stop

Q.2 Explain primary memory and secondary storage. [R.T.U. 2019]

Ans. Primary memory v/s secondary memory

A computer contains a hierarchy of memory devices for storing data. They vary in their capacity, speed and cost. Primary memory (also referred to as the main memory) is the memory that is directly accessed by the CPU to store and retrieve information. Secondary memory (also referred to as the external or auxiliary memory) is a storage device that is not accessible directly by the CPU and used as a permanent storage device that retains data even after the power is turned off.

Primary memory is the memory that is directly accessed by the CPU to store and retrieve information. Most of the time, primary memory is also referred to as the RAM (Random Access Memory). It is a volatile memory, which loses its data when the power is turned off. Primary memory is directly accessible by the CPU through the address and memory bus and it is constantly accessed by the CPU to get data and instructions. Furthermore, computers contain a ROM (Read Only

Memory), which holds instructions that are executed often such as the startup program (BIOS). This is a non volatile memory that retains its data when the power is turned off. Since the main memory is accessed often, it needs to be faster. But they are smaller in size and also costly.

Secondary memory is a storage device that is not accessible directly by the CPU and used as a permanent storage device that retains data even after power is turned off. CPU accesses these devices through an input/output channel and data is first transferred in to the primary memory from the secondary memory before accessing. Usually, hard disk drives and optical storage devices (CDs, DVDs) are used as secondary storage devices in modern computers. In a secondary storage device, data are organized in to files and directories according to a file system. This also allows to associate additional information with data such as the access permissions, owner, last access time, etc. Furthermore, when the primary memory is filled up, secondary memory is used as a temporary storage for keeping least used data in the primary memory. Secondary memory devices are less costly and larger in size. But they have a large access time.

However, primary memory is the memory that is directly accessed by the CPU to store and retrieve information, whereas the secondary memory is not accessible directly by the CPU. Primary memory is accessed using address and data buses by the CPU, while secondary memory is accessed using input/output channels. Primary memory does not retain data when the power is turned off (volatile) while secondary memory retains data when the power is turned off (non-volatile). Furthermore, primary memory is very fast compared to the secondary memory and has a lower access time. But, primary memory devices are more costly compared to secondary memory devices. Due to this reason, usually a computer comprises of a smaller primary memory and a much larger secondary memory.

Q.3 Write the difference between random access and sequential access method. [R.T.U. 2019]

Ans. Random Access Method : In computer science, **random access** (sometimes called **direct access**) is the ability to access an arbitrary element of a sequence in equal time. The opposite is sequential access, where a remote element takes longer time to access. A typical illustration of this distinction is to compare an ancient scroll (sequential; all material prior to the data needed must be unrolled) and the book (random: can be immediately flipped open to any random page). A more

modern example is a cassette tape (sequential—you have to fast-forward through earlier songs to get to later ones) and a compact disc (random access—you can jump right to the track you want). The term Random Access Memory (RAM), however, is used for semiconductor chip memory circuits used in computers. (The term was also used to describe ferrite-core memory in early computers).

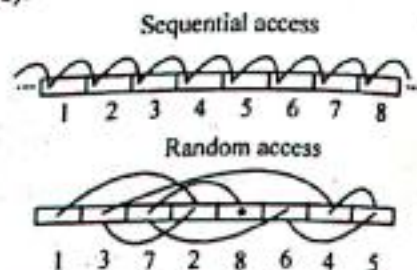


Fig. : Random Access Compared to Sequential Access

In data structures, random access implies the ability to access the n^{th} entry in a list of numbers in constant time. Very few data structures can guarantee this, other than arrays (and related structures like dynamic arrays). Random access is critical to many algorithms such as quick sort and binary search. Other data structures, such as linked lists, sacrifice random access to make for efficient inserts, deletes, or reordering of data.

Sequential Access Method

In computer science, **sequential access** means that a group of elements (e.g. data in a memory array or a disk file or on a tape) is accessed in a predetermined, ordered sequence. Sequential access is sometimes the only way of accessing the data, for example if it is on a tape. It may also be the access method of choice, for example if we simply want to process a sequence of data elements in order.

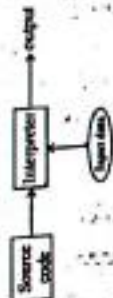
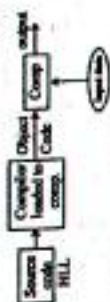
In data structures, a data structure is said to have sequential access if one can only visit the values it contains in one particular order. The canonical example is the linked list. Indexing into a list which has sequential access requires $O(k)$ time, where k is the index. As a result, many algorithms such as quicksort and binary search degenerate into bad algorithms that are even less efficient than their alternatives; these algorithms are impractical without random access. On the other hand, some algorithms, typically those which don't perform indexing, require only sequential access, such as mergesort, and so face no penalty.

Q.4 Write the difference between compiler and interpreter. [R.T.U. 2019]

Programming for Problem Solving

Ans.

Assembler	Compiler	Interpreter
1. Assembler is a translator that translates the assembly language program into the m/c language.	1. Complex and difficult to write	1. Simple and easy to write
	2. Require large memory space to store itself	2. Doesn't require much memory space to store itself.
	3. Takes more time in translation.	3. Take less time in translation
	4. Translates the entire program into m/c language at once.	4. Translates the program line by line into m/c language
2. It also assembles the m/c code in memory of computer and makes the program ready for execution.	5. Checks the syntax errors in program when whole program become error free then it is translated into m/c code and executed	5. Check for the syntax errors line by line and executes each Line immediately if statement is error free.
	6. Takes less time to execute program	6. Takes more time to execute programs
	7. Slow for debugging	7. Fast for debugging



Q.5 Write the difference between High-Level language and Low-Level language. [R.T.U. 2018]

Ans. High level v/s Assembly level programming :

	Machine language	Assembly language	High level language
1.	It is the language i.e. understands by the computer without using a translation program.	In assembly language we use mnemonic for numeric opcodes of m/c language	High level language is basically problem oriented. While writing program in the HLL a programmer use commands generally written using English words and mathematical symbols. HLL programs are translated into machine language by means of a system program called compilers.
2.	This is also called as m/c code of the computer and is normally written as string of binary 0's and 1's.	Computer can translate each Line of assembly language Program into m/c language. Program by means of translating program.	
3.	Advantages Program in m/c language can be executed very fast.	Advantages Easy to understand and use easier to modify & relocatable.	Advantages Machine independent fewer errors and easy debugging.
4.	Disadvantages 1. Machine dependent. 2. Difficult to program and modify. 3. Time consuming and error prone.	Disadvantages 1. Machine dependent. 2. Knowledge of h/w required.	Disadvantages Speed (HLL program takes more time and memory to run).

Q.6 Write short note on Pseudo Code. [R.T.U. 2015, 13]

Ans. Pseudocode : Pseudocode is made up on two words 'Pseudo' and 'Code'. Pseudo means 'false' or

'unitation' and code means 'instruction' written in a programming language. As the name suggests, pseudocode is not a real programming code, but it models programming code. It is generic way of describing an algorithm without using any specific programming language constructs. Pseudocode uses plain english statement rather than symbols to represent the process of a computer program.

Advantages of Pseudocode

- Since it is language independent, it allows the developer to express the design in plain natural language.
- Unlike flowcharts, pseudocode is compact and does not tend to run over many phases. Its simple structure and readability makes it easier to modify as well.

Limitations

- The main limitation of pseudocode is that it does not provide visual representation of the program's logic.
- There are no accepted standards for writing pseudocodes. Different programmers use their own style of writing pseudocode.

Q.7 Write short note on Flow chart and algorithms. (R.T.U. 2015)

Ans. Flowchart : A flowchart is a pictorial representation of an algorithm in which the steps are drawn in the form of different shapes of boxes and the logical flow is indicated by interconnecting arrows. The boxes represent operations and the arrows represent the sequence in which the operations are implemented. The primary purpose of the flowchart is to help the programmer in understanding the logic of the program. Therefore, it is always not necessary to include all the required steps in detail. Flowcharts outline the general procedure. Since they provide an alternative, visual way of representing the information flow in a program, program developers often find them very valuable.

Flowcharts can be compared with the blueprint of a building. Just as an architect draws a blueprint before starting the construction of a building, a programmer draws a flowchart before writing a computer program. As in the case of the drawing of a blueprint, the flowchart is drawn according to defined rules and using standard flowchart symbols prescribed by American National Standard Institute (ANSI).

Algorithms : Refer to Q.1.

Q.8 Write short note on assembly language.

(R.T.U. 2013)

Ans. Assembly Language : The complexities of the machine language led to the development of Assembly Language which is called second generation of programming languages. Unlike other programming languages, assembly language is not a single language, but a group of languages. Each processor family has its own assembly language. Thus, assembly language is different for computers of different architectures.

The assembly language assigns a mnemonic code to each machine language instruction to make it easier to remember or write. It allows better human readable method of writings programs as compared to writing in binary bit patterns.

Assembly language provides mnemonics i.e., is easy to remember short names, for the various operations supported by the machine language. For example, ADD is used to perform addition operation, MULT is used for multiplication etc. The assembly language also allows us to refer to memory locations by symbolic names. Thus an assembly language program consists of symbols or names instead of numeric codes.

Apart from the use of mnemonics and symbolic names, the assembly language instructions have one to one correspondence with the machine language instructions. A sample machine language instruction and its equivalent assembly language instruction.

1	0	1	0	0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---

Operation code

Address of a memory location

(a) A sample machine language instructions

ADD

NUM

Mnemonic, which has replaced the opcode in machine instruction.

A symbolic name which has replaced the address of a location in the machine instruction.

(b) An equivalent Assembly language instruction

As you are aware of that a computer can understand only machine language, hence the programs written in assembly language have to be converted into machine language for execution. A program that does this translation is called an **assembler**. The assembler recognizes the assembly language mnemonics and symbolic names of memory locations and converts them into appropriate binary code.

Q.9 What do you mean by wordlength of a processor.
[R.T.U. 2012]

Ans. Wordlength is the number of bits that a CPU can process at once. A processor with a 32-bit wordlength has the capacity to be twice as fast as a processor with a 16 bit wordlength. Today's personal computers typically use wordlengths of 32 or 64 bits. Intel Pentium 4 processors have a 32 bit wordlength; AMD Athlon 64 processors have a word length of 64 bits. The benefits of 64 bit wordlength are only experienced if the operating system and software are designed to take advantage of the processor's 64-bit capabilities. While Intel processors and both Windows and Mac OS X have evolved to 64 bit architectures, software developers have been slower to migrate their applications from 32-bit to 64-bit. That is likely to change as 64-bit processors become the norm.

PART B

Q.10 Write a Pseudo code to multiply $2[3 \times 3]$ Matrices and Transpose the output of multiplication.
[R.T.U. 2019]

```
Ans. #include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,m,n,p,q,k,sum=0; /* Declaring sum=0 since we
    use it as a temporary variable while multiplication */
    int first[3][3], second[3][3], multiply[3][3], transpose
    [3][3];
    /* Accepting 1st matrix */
    printf("Enter the number of rows and columns of
    first matrix\n");
    scanf("%d%d",&m,&n);
    printf("Enter the elements of first matrix\n"); /* Order
    (m,n) */
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&first[i][j]);
```

```

        }
    }
    /* Accepting 2nd Matrix */
    printf("Enter the number of rows and columns of
    second matrix\n");
    scanf("%d%d",&p,&q);
    if(n!=p) /* To multiply matrices, Number of columns
    of 1st matrix should be equal to number of rows in 2nd
    matrix */
    printf("matrix with entered order can't be
    multiplied with each other.\n");
    else /* Accepting 2nd Matrix */
    {
        printf("Enter the element of second matrix\n");
        /* Order (p,q) */
        for(i=0;i<p;i++)
        {
            for(j=0;j<q;j++)
            {
                scanf("%d",&second[i][j]);
            }
        }
    }
    /* Matrix Multiplication */
    for(i=0;i<m;i++)
    {
        for(j=0;j<q;j++)
        {
            for(k=0;k<p;k++)
            {
                sum=sum+first[i][k]*second[k][j];
            }
            /* calculating and storing final value of element in
            sum */
            multiply[i][j]=sum; /* Assigning value stored in sum
            to Matrix: multiply[i][j] */
            sum=0; /* Again we will make sum=0 since we
            want to multiply next element */
        }
    }
    /* Printing multiplied matrix */
    printf("Product of entered matrices\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<q;j++)
        {
            printf("%d\t",multiply[i][j]);
```

```

    }
    printf("\n");
}
/* transpose of matrix */
for(i=0;i<m;i++)
{
    for(j=0;j<n;j++)
    {
        transpose[i][j]=multiply[j][i];
        /*for example i=1 and j=2 then in matrix, element
        (1,2) will be swapped with (2,1) thus we get transpose of
        matrix:*/
    }
}
/* Printing transposed matrix */
printf("Transpose of the matrix \n");
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        printf("%d\t",transpose[i][j]);
    }
    printf("\n"); /* \n to bring cursor to next line and
    print elements in matrix format */
}
getch();
}

```

Q.11 Draw the block diagram of a Computer. Explain its components. Differentiate between primary and secondary storage. [R.T.U. Dec. 2019]

OR

Draw a block diagram of basic architecture of a computer system. [R.T.U. May 2019]

Ans. The size, shape, performance, reliability, and cost of computers have been changing over the years. The basic logical structure (based on the stored program concept), as proposed by Von Neumann, has not changed. No matter what shape and size of computer we are talking about:

All computer systems perform the following five basic operations, for converting raw input data into information which is useful to their users:

1. Inputting : The process of entering data and instructions into the computer system.

2. Storing : Saving data and instructions to make them readily available for initial or additional processing, as and when required.

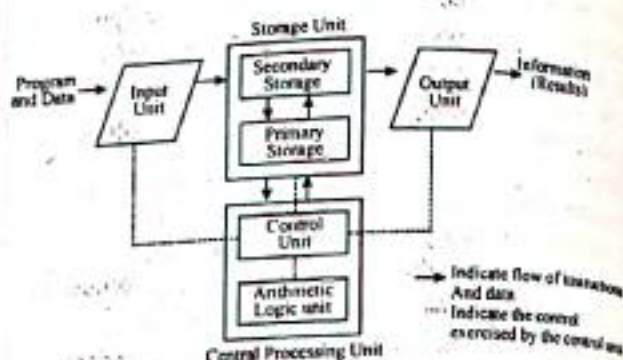


Fig. : Architecture of a computer system

3. Processing : Performing arithmetic operations (add, subtract, multiply, divide, etc.), or logical operations (comparisons like equal to, less than, greater than, etc.) on data, to convert them into useful information.

4. Outputting : The process of producing useful information or result for the user, such as a printed report or visual display.

5. Controlling : Directing the manner and sequence in which all of the above operations are performed.

The internal architecture of computers differs from one system model to another. However, the basic organization remains the same for all computer systems. A block diagram of the basic computer organization is shown in figure. In the given figure, the solid lines indicate the flow of instruction and data, and the dotted lines represent the control exercised by the control unit. It displays the five major building blocks (functional units) of a digital computer system. These five units correspond to the five basic operations, performed by all computer systems.

Input Unit : Data and instructions must enter the computer system, before any computation is performed on the supplied data. This task is preformed by the input unit, which links the external environment with the computer system. Data and instructions enter input units in forms, which depend upon the particular device used. For example, data are entered from a keyboard in a manner similar to typing, and this differs from the way in which data are entered through a scanner, which is another type of input device. However, regardless of the form in which they receive their inputs, all input devices

must transform the input data into the binary codes, which the primary memory of a computer is designed to accept. This transformation is accomplished by units called input interfaces. Input interfaces are designed to match the unique physical or electrical characteristics of input device to the requirements of the computer system. In short, an input unit performs the following functions:

1. It accepts (or reads) the instructions and data from the outside world.
2. It converts these instructions in computer acceptable form.
3. It supplies the converted instructions and data to the computer system for further processing.

Output Unit : The job of an output unit is just the reverse of that of an input unit. It supplies the information obtained from data processing, to the outside world. Hence, it links the computer with the external environment. As computers work with binary code, the results produced are also in the binary form. Hence before supplying the result to the outside world, they must be converted to human acceptable (readable) form. This task is accomplished by units called output interfaces. Output interfaces are designed to match the unique physical or electrical characteristics of output devices (terminals, printers, etc.), to the requirements of the external environment.

In short, an output unit performs the following functions :

1. It accepts the results produced by the computer, which are in coded form, and hence cannot be easily understood by us.
2. It converts these coded results to human acceptable (readable) form.
3. It supplies the converted result to the outside world.

Storage Unit : The data and instructions, which are entered into the computer system through input units, have to be stored inside the computer, before the actual processing starts. Similarly, the results produced by the computer after processing, must also be kept somewhere inside the computer system, before being passed on to the output units. Moreover, the intermediate results produced by the computer, must also be preserved for ongoing processing. The storage unit of a computer system is designed to cater to all these needs. It provides space for storing data and instructions, space for intermediate results, and space for the final results.

In short, the specific functions of the storage unit are to hold (store):

1. The data and instructions required for processing (received from input devices).
2. Intermediate results of processing.
3. Final results of processing, before these results are released to an output device.

The storage unit of all computers is comprised of the following two types of storage :

1. Primary Storage : The primary storage, also known as main memory, is used to hold pieces of program instructions and data, intermediate results of processing, and recently produced results of processing, of the job(s), which the computer system is currently working on. These pieces of information are represented electronically in the main memory chip's circuitry, and while it remains in the main memory, the central processing unit can access it directly at a very fast speed. However, the primary storage can hold information only while the computer system is on. As soon as the computer system is switched off or reset, the information held in the primary storage disappears. Moreover, the primary storage normally has limited storage capacity, because it is very expensive. The primary storage of modern computer system is made up of semiconductor devices.

2. Secondary Storage : The secondary storage, also known as auxiliary storage, is used to take care of the limitations of the primary storage. That is, it is used to supplement the limited storage capacity and the volatile characteristic of primary storage. This is because secondary storage is much cheaper than primary storage, and it can retain information even when the computer system is switched off or reset. The secondary storage is normally used to hold the program instructions, data, and information of those jobs, on which the computer system is not working on currently, but needs to hold them for processing later. The most commonly used secondary storage medium is the magnetic disk.

Central Processing Unit (CPU) : The Control Unit and the Arithmetic Logic Unit of a computer system are jointly known as the Central Processing Unit (CPU). The CPU is the brain of a computer system. In a human body, all major decisions are taken by the brain, and the other parts of the body function are directed by the brain. In a computer system, all major calculations and comparisons are made inside the CPU, and the CPU is responsible for activating and controlling the operations of other units of the computer system.

(i) **Arithmetic Logic Unit** : The Arithmetic Logic Unit (ALU) of a computer system is the place, where the actual execution of the instructions takes place, during the processing operation. To be more precise, calculations are performed, and all comparisons (decision) are made in the ALU. The data and instructions, stored in the primary storage before processing, are transferred as and when needed to the ALU, where processing takes place. No processing is done in the primary storage unit. Intermediate results generated in the ALU are temporarily transferred back to the primary storage, until needed later. Hence, data may move from primary storage to ALU, back again to storage, many times, before the processing is over.

The type and number of arithmetic and logic operations, which a computer can perform, is determined by the engineering design of the ALU. However, almost all ALUs are designed to perform the four basic arithmetic operations (add, subtract, multiply, and divide), and logic operations or comparisons (such as less than, equal to, and greater than).

(ii) **Control Unit** : How does the input device know that it is time for it to feed data into the storage unit? How does the ALU know, what should be done with the data once they are received? Moreover, how is it that only the final results are sent to the output device, and not the intermediate results? All this is possible due to the control unit of the computer system. Although, it does not perform any actual processing on the data, the control unit acts as a central nervous system, for the other components of the computer system. It manages and coordinates the entire computer system. It obtains instructions from the program stored in main memory, interprets the instructions, and issues signals, which cause other units of the system to execute them.

Q.12 Write short notes on :

- System software
- Compiler
- Freeware software

[R.T.U. 2018]

Ans.(a) System Software : System software is a type of computer program that is designed to run a computer's hardware and application programs. If we think of the computer system as a layered model, the system software is the interface between the hardware and user applications.

The operating system (OS) is the best-known example of system software. The OS manages all the other programs in a computer. Other examples of system software and what each does:

- The BIOS (basic input/output system) gets the computer system started after you turn it on and manages the data flow between the operating system and attached devices such as the hard disk, video adapter, keyboard, mouse, and printer.
- The boot program loads the operating system into the computer's main memory or Random Access Memory (RAM).
- An assembler takes basic computer instructions and converts them into a pattern of bits that the computer's processor can use to perform its basic operations.
- A device driver controls a particular type of device that is attached to your computer, such as a keyboard or a mouse. The driver program converts the more general input/output instructions of the operating system to messages that the device type can understand.

According to some definitions, system software also includes system utilities, such as the disk defragmenter and system restore, and development tools such as compilers and debuggers.

Ans.(b) Compiler : A compiler is a kind of translator that translates a program written in a high level language into machine code. The compiler replaces single high level statement into a series of machine language instructions. During the translation process, the compiler reads the source program and checks the syntax (grammatical) errors. If there is any error, the compiler generates an error message which is displayed on the screen. In case of errors, the compiler doesn't create the object code until all the errors are rectified.

Once the source code has been compiled successfully, the compiler saves the resulting machine code in a file separately. This makes recompiling necessary only if the source code has been modified.

The working of a compiler can be depicted as:



Fig. : Working of a Compiler

Ans.(c) Freeware Software : Freeware is computer software that is made available free of charge, but which is copyrighted by its developer, who retains the rights to control its distribution, modify it and sell it in the future. It is typically distributed without its source code, thus preventing modification by its users.

This can be freely used with only one restriction: any redistributed version of the software must be distributed with the original terms of free use, modification, and distribution (known as copyleft). Freeware is usually distributed with a license that permits its redistribution to some extent, for example allowing users to give copies to friends. Some licenses permit the software to be freely copied but not sold.

Q.13 Give classification for the types of language working in computer programming environment.
[R.T.U. 2015]

Ans. Computers understand only one language and that is binary language or the language of '0's and '1's. Binary language is also known as machine language. In the initial years of computer programming, all the instructions were given in binary form only. Although these programs were easily understood by the computer, it proved too difficult for a normal human being to remember all the instructions in the form of '0's and '1's. Therefore, the computer remained a mystery to a common person until other languages, such as assembly and high-level languages, were developed which were easier to learn and understand. These languages use commands that have some degree of similarity with English (such as "if else," "exit"). Programming languages can be divided into three major categories:

1. Machine Language : It is the native language of computers. It uses only '0's and '1's to represent data and the instructions.

2. Assembly Language: It corresponds to symbolic instructions and executable machine codes and was created to use letters instead of '0's and '1's to run a machine.

3. High-level Language: These languages are written using a set of words and symbols following some rules similar to a natural language, such as English. The programs written in high-level languages are known as source programs and these programs are converted into machine-readable form by using compilers or interpreters.

Note : Together, machine and assembly language are also known as low-level languages.

Q.14 Write differences between the following :

(i) Interpreter and compiler

(ii) High level, assembly and machine language.

[R.T.U. 2014]

OR

Machine, Assembly and High Level Language

[R.T.U. 2011]

Ans.(i) Refer to Q.4.

(ii) Refer to Q.5.

Q.15 Differentiate between random access method and sequential access method for memory devices.
[R.T.U. 2014, 2012]

Ans. Random Access Method : Refer to Q.3.

Q.16 (a) What is pseudocode ? Write the advantage and limitation of pseudocode. [R.T.U. 2010]

(b) Write short note on volatile memory.

[R.T.U. 2013]

OR

Write short note on direct access memory.

[R.T.U. 2013]

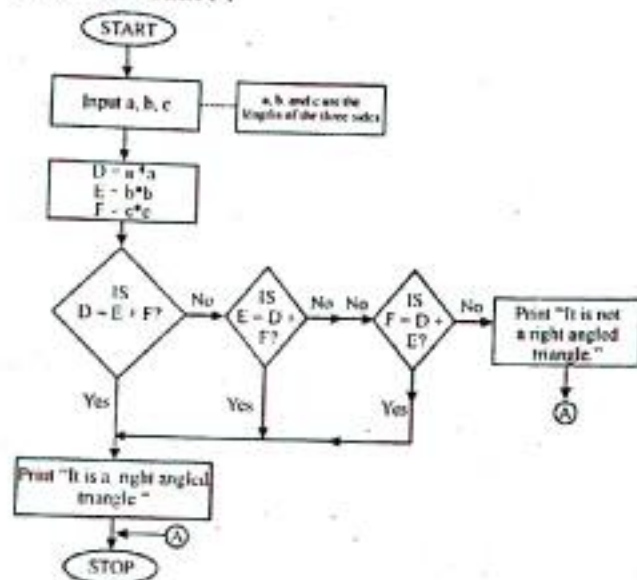
Ans.(a) Pseudocode : Refer to Q.6.

Ans.(b) Volatile Memory : It requires constant power to maintain the stored information. The fastest memory technologies of today are volatile ones. Since primary storage is required to be very fast, it predominantly uses volatile memory.

Random Access Memory (RAM) : RAM is volatile in nature, this memory consists of some integrated circuit chips either on motherboard, or on a small circuit board attached to the motherboard. A computer's motherboard is designed in a manner that its memory capacity can be easily enhanced by adding more memory chips. If you require to insert more memory, then you can easily plug-in extra memory in the empty memory slots on the motherboard. A computer containing only primary memory would not have a source to read instructions, in order to start the computer. Hence, non-volatile primary memory containing a small startup program (BIOS) is used to bootstrap the computer, that is, to read a larger program from non-volatile secondary memory to RAM and start to execute it.

Q.17 Draw a flow chart which takes as input length of three sides of a triangle and check if it is a right angled triangle. [R.T.U. 2012]

Ans. Flow chart :



Q.18 (a) Give reasons why assembly languages and high level languages were designed while computer process only machine language.
(b) Draw a flow chart which reads N numbers and computes average of these numbers. [R.T.U. 2012]

Ans.(a) In the computer, all data, whatever its form, is stored and used in binary codes—that is, as strings of 1s and 0s. Instructions and data are stored together in the computer's memory using these binary codes. If you were to look at the binary codes representing instructions and data in memory, you could not tell the difference between them; they are distinguished only by the manner in which the computer used them. This scheme makes it possible for the computer to process its own instructions as a form of data.

When computers were first developed, the only programming language available was the primitive instruction set built into each machine, the machine language or machine code.

Even though most computers perform the same kinds of operations, their designers choose different sets of binary codes for each instruction. Thus the machine code for one computer is not the same as the machine code for another computer.

When programmers used machine language for programming, they had to enter the binary codes for the various instructions, a tedious process that was prone to error. Moreover, their programs were difficult to read and modify. In time, assembly language were developed to make the programmer's job easier.

Instructions in an assembly language are in an easy-to-remember form called a mnemonic (pronounced nee-MAHN-ik). Typical instructions for addition and subtraction might look like this :

Assembly language	Machine language	Meaning
ADD R1, R3, R5	1001 001 011 010	Add R3 to R5 and put result in R1
SUB R2, R6, R1	1010 010 110 001	Subtract R1 from R6, result in R2
Operand Result Operand 1 Operand 2	Operand Result Operand 1 Operand 2	

Although assembly language is easier for humans to work with, the computer cannot directly execute the instructions. One of the fundamental discoveries in computer science is that, because a computer can process its own instructions as a form of data, it is possible to write a program to translate the assembly language instructions into machine code.

Such a program is being referred as assembly language program. The name comes from the fact that much of what an assembler does is to look up the pieces of an instruction in a table to find the corresponding binary code (such as ADD = 1001, R1 = 001, R3 = 011, R5 = 101) and then assemble these binary coded pieces of the instruction into a complete machine language instruction (1001 001 011 101). The assembler also puts the instructions together in the specified sequence to create a complete program.

Assembly language is a step in the right direction, but it still forces programmers to think in terms of individual machine instructions. Eventually, computer scientists developed high-level programming languages. These language are easier to use than assembly language or machine code because they are closer to english and other natural language.

A program called a compiler translate programs written in certain high-level languages (C++, Java, Visual Basic, Ada, etc.) into machine language. If you write a program in a high-level language, you can run it on any computer that has the appropriate compiler. This is possible because most high-level languages are standardized, which means that an official description of the language exists.

A program in a high-level language is called a source program. To the compiler, a source program is just input data. It translates the source program into a

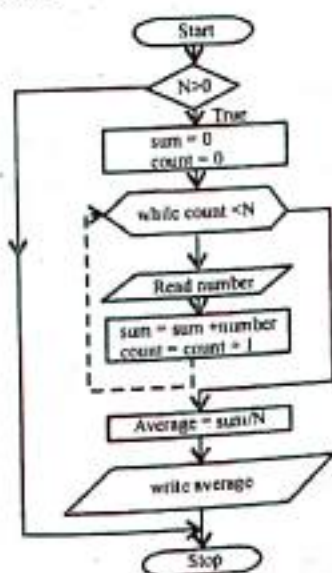
Programming for Problem Solving

machine language program called an object program. If there are errors in the program, the compiler instead generates one or more error message indicating the nature and location of the errors.

One benefit of using standardized high-level language is that they allow you to write portable (or machine-independent) code. A single C++ program can be used on different machines, whereas a program written in assembly language or machine language is not portable from one computer to another. Because each computer has its own machine language, a machine language program written for computer A will not run on computer B.

It is important to understand that compilation and execution are two distinct processes. During compilation, the computer runs the compiler program. During execution, the object program is loaded into the computer's memory unit, replacing the compiler program. The computer then runs the object program, doing whatever the program instructs it to do.

Ans. (b) In the following flow chart N is the integer variable denoting the number of values considered for computing the average count is another integer variable denoting the number of values that are processed at any instant. The number is an integer variable for storing the values.



Q.19 What do you mean by Language Translators?
[R.T.U. 2011]
Explain with example.

Ans. Language Translators : A language translator is a program that accepts a program written in a language (other than machine language) also called source code

and converts it into an equivalent machine language code called as target code.

Basically there are three types of language translators :

1. Assembler 2. Compiler 3. Interpreter

(1) **Assembler** : Assembly language is nothing but a symbolic representation of machine code. But inspite of this closeness between assembly and machine code, the computer still cannot understand it. The assembly language program is translated into machine code by a separate program called an assembler. The assembler program recognizes the character string that make up the symbolic names of the various machines operations and substitutes the require machine code for each instruction. At the same time, it also calculates the required address in memory for each symbolic name of memory location and substitutes those addresses for the names resulting in machine language program that can run on the computer. We can summarize the function of an assembler as -

1. It allows the programmer to use mnemonics while writing source code programs, which are easier to read and follow.
2. It allows the variables to be represented by symbolic names, not as memory locations.
3. It checks the syntax of the assembly program and generates diagnostic message on syntax errors.
4. It translates assembly language instructions into machine language.
5. It assembles all the instructions in the main memory for execution.

The working of an assembler can be depicted as :



Fig. 1 : Working of an Assembler

(2) **Compiler** : Refer to Q.12(b).

(3) **Interpreter** : An interpreter is also a language translator which translates a high level program into machine language. However, unlike compiler, it translates a statement of a high level language program and executes the statement immediately i.e., before translating the next source language statement. When a syntax error is encountered in the program, the execution of the program is halted and an error message is displayed. Some languages that have interpreters are : BASIC, LISP etc.

The functioning of an interpreter can be shown as:



Fig. 2 : Working of an Interpreter

Q.20 Differentiate between ROM and PROM.

[R.T.U. 2011, Raj. Univ. 2002]

Ans. Difference between ROM and PROM

ROM (Read Only Memory) : A special type of RAM, called Read Only Memory (ROM), is a non-volatile memory chip, in which data is stored permanently and cannot be altered by the programmer. In fact, storing data permanently into this kind of memory is called "burning in the data", because data in such memory is stored by using fuse-links. Once a fuse-link is burnt, it is permanent. The data stored in a ROM chip can only be read and used, they cannot be changed. This is the reason why it is called Read Only Memory (ROM). Since ROM chips are non-volatile, the data stored inside a ROM are not lost when the power supply is switched off, unlike the case of a volatile RAM chip. ROMs are also known as field stores, permanent stores, or dead stores.

ROMs are mainly used to store program and data, which do not change and are frequently used. For example, the most basic computer operations are carried out by wired electronic circuits. However, several higher-level operations, which are frequently used, require very complicated circuits for their implementations.

Hence, instead of building electronic circuits for these operations, special programs are written to perform them. These programs are called micro programs, because they deal with low-level machine functions and are essentially substitutes for additional hardware. ROMs are used by computer manufacturers for storing these micro programs, so that they cannot be modified by the users.

PROM (Programmable Read Only Memory) : Similar to Read Only Memory with the exception that these chips can be reprogrammed by using special external equipments. There are two type of Read Only Memory (ROM)-manufacturer-programmed and user-programmed.

A **manufacturer-programmed ROM** is one in which data is burnt in by the manufacturer of the electronic equipment in which it is used. For example, a personal computer manufacturer may store the system

boot program permanently in the ROM chip used on the motherboard of all the PCs manufactured by it. Similarly, a printer manufacturer may store the printer controller software in the ROM chip used on the circuit board of all the printers manufactured by it. **Manufacturer-programmed ROMs** are mainly used in those cases, where the demand for such programmed ROMs is large. Note that manufacturer programmed ROM chips are supplied by the manufacturers of electronic equipment, and it is not possible for a user to modify the programs or data stored inside the ROM chip. On the other hand, a **user-programmed ROM** is one in which the user can load and store "read only" programs and data. That is, it is possible for a user to "customize" a system by converting his/her own programs to micro programs, and storing them in a user-programmed ROM chip. Such a ROM is commonly known as **Programmable Read Only Memory (PROM)**, because a user can program it. Once the user programs are stored in a PROM chip, they can usually be executed in a fraction of the time previously required. PROMs are programmed to record information using a special device, known as PROM-programmer. However, once the chip has been programmed, the recorded information cannot be changed, i.e., the PROM become a ROM, and it is only possible to read the stored information. PROM is also non-volatile storage, i.e., the stored information remains intact, even if power is switched off.

Q.21 Write short note on different types of RAMs.

[R.T.U. 2010, Raj. Univ. 2003]

Ans. Random Access Memory (RAM) : Refer to Q.16(b).

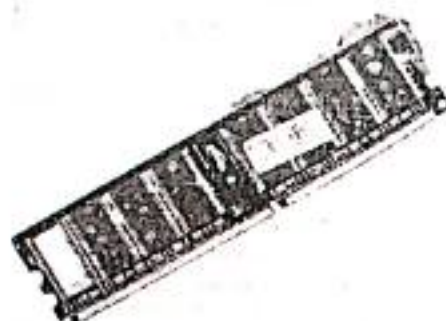


Fig. : RAM Chip

Types of RAM :

Primary Memory : Random Access Memory (RAM) and Read Only Memory (ROM) fall in the category of

the primary memory. Every computer comes with a small amount of ROM, which contains the boot firmware (called BIOS). This holds just enough information so that the computer can check its hardware and load its operating system into RAM.

RAM is the place where the computer temporarily stores its operating system, application programs, the current data, so that the computer's processor can reach them quickly and easily. It is volatile in nature, i.e. when the power is switched off, the data in the memory is lost. Unlike RAM, ROM is non-volatile. Even when the computer is switched off, the contents of ROM remain available.

Types of RAM : RAM can be further divided into two categories :

(i) **Static Random Access Memory (SRAM)** is a type of semiconductor memory that does not need to be periodically refreshed like DRAM. SRAM is considerably faster than DRAM. It is effective because most of the programs access the same data repeatedly and keeping all this information in the fast SRAM that allows the computer to avoid accessing the slow DRAM. SRAM exhibits remembrance, but is still volatile in the conventional sense that data is eventually lost when the memory is not powered.

(ii) **Dynamic Random Access Memory (DRAM)**: It is a type of RAM that stores each bit of data in a separate capacitor within an integrated circuit. Since real capacitor leak charge, the information eventually fades unless the capacitor charge is refreshed periodically. Because of this refresh requirement, it is a dynamic memory as opposed to SRAM and other static memory. Like SRAM, it is in the class of volatile memory devices, since it loses its data when the power supply is removed. Unlike SRAM however, data may still be recovered for a short time after power-off.

Types of ROM -

Programmable read-only memory (PROM), or onetime programmable ROM (OTP), can be written to or programmed via a special device called a PROM programmer. Typically, this device uses high voltages to permanently destroy or create internal links (fuses or antifuses) within the chip. Consequently, a PROM can only be programmed once.

Erasable programmable read-only memory (EPROM) can be erased by exposure to strong ultraviolet light (typically for 10 minutes or longer), then rewritten with a process that again requires application of higher than usual voltage. Repeated exposure to UV light will eventually wear out an EPROM, but the

endurance of most EPROM chips exceeds 1000 cycles of erasing and reprogramming. EPROM chip packages can often be identified by the prominent quartz "window" which allows UV light to enter. After programming, the window is typically covered with a label to prevent accidental erasure.

Electrically erasable programmable read-only memory (EEPROM) is based on a similar semiconductor structure to EPROM, but allows its entire contents (or selected banks) to be electrically erased, then rewritten electrically, so that they need not be removed from the computer (or camera, MP3 player, etc.). Writing or flashing an EEPROM is much slower (milliseconds per bit) than reading from a ROM or writing to a RAM (nanoseconds in both cases).

Electrically alterable read-only memory (EAROM) is a type of EEPROM that can be modified one bit at a time. Writing is a very slow process and again requires higher voltage (usually around 12 V) than is used for read access. EAROMs are intended for applications that require infrequent and only partial rewriting. EAROM may be used as non-volatile storage for critical system setup information; in many applications, EAROM has been supplanted by CMOS RAM supplied by mains power and backed-up with a lithium battery.

Flash memory (or simply flash) is a modern type of EEPROM invented in 1984. Flash memory can be erased and rewritten faster than ordinary EEPROM, and newer designs feature very high endurance (exceeding 1,000,000 cycles). Modern NAND flash makes efficient use of silicon chip area, resulting in individual ICs with a capacity as high as 16 GB as of 2007[update]; this feature, along with its endurance and physical durability, has allowed NAND flash to replace magnetic in some applications (such as USB flash drives). Flash memory is sometimes called flash ROM or flash EEPROM when used as a replacement for older ROM types, but not in applications that take advantage of its ability to be modified quickly and frequently.

Q.22 Discuss various types of memory. Explain primary and secondary memory in detail.

[R.T.U. 2010, 2008]

Ans. Memory refers to the electronic holding place for instructions and data. CPU requires memory to handle the initial input, intermediate and final results. We can classify memory into two basic categories :

- (i) Primary memory
 - (ii) Secondary memory
-

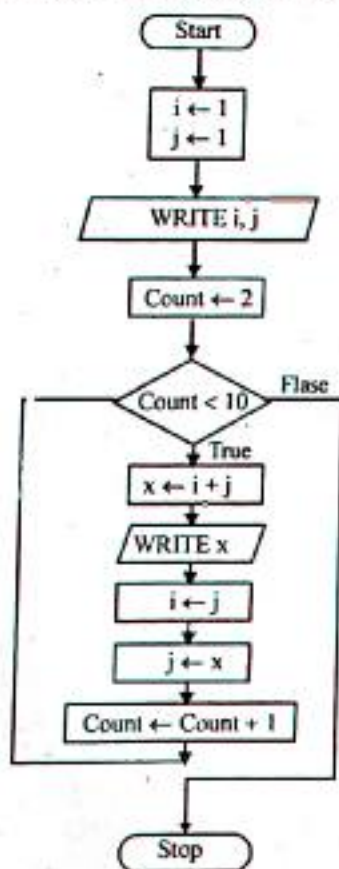
(i) **Primary Memory** : Refer to Q.21.

(ii) **Secondary Memory** : It is also known as auxiliary memory. Secondary memory provides backup storage for instructions (software programs) and data. Most commonly used secondary storage devices are Hard Disk, Magnetic Disk and Magnetic Tapes. These are the least expensive among all the memories. However, they have much larger storage capacity than primary memory. Instructions and data stored on such devices are permanent in nature. It can only be removed if the user wants it or the device is destroyed the benefits of secondary storage are :

- (i) Non volatility
- (ii) Reliability
- (iii) Convenience
- (iv) Less expensive
- (v) Reusability
- (vi) Portability

Q.23 Draw a flow chart for computing first 10 fibonacci number. [R.T.U. 2009]

Ans. Flow Chart of Fibonacci Number



PART C

Q.24 Write short notes on following :

- (a) Types of Primary Memory
- (b) High level v/s Assembly level programming
- (c) Notations of Flow chart
- (d) Primary memory v/s secondary memory

[R.T.U. 2014]

Ans.(a) Primary Memory : Refer to Q.21.

Ans.(b) High level v/s Assembly level programming : Refer to Q.5.

Ans.(c) Notations of Flowchart

There are rules and standards (notations) for drawing flowcharts, prescribed by American National Standard Institute (ANSI). Table shows some standard symbols which are frequently required in the flow charts.

Symbol	Symbol Name	Description
	Flow lines	Flow lines are used to connect symbols used in flow-chart and indicate direction of flow.
	Terminal (START / STOP)	This is used to represent start and end of the flow-chart.
	Input / Output	It represent information which the system reads as input or sends as output.
	Processing	Any process is represented by this symbol. For example arithmetic operation, data movement.
	Decision	This symbol is used to check any condition or take decision for which there are two answers. Yes (True) or No (False).

	Connector	It is used to connect or join flow lines.
	Off-page Connector	This symbol indicates the communication of flowchart on the next page.
	Document	It represents a paper document produced during the flowchart process.
	Annotation	It is urged to provide additional information about another flowchart symbol which may be in the form of descriptive comments, remark or explanatory notes.
	Manual Input	It represents input to be given by a developer or programmer.
	Manual operation	This symbol indicates that the process has to be done by a developer or programmer.
	Online storage	It represents online data storage such as hard disks, magnetic drums or other storage devices.
	Offline storage	It represents the offline data storage such as sales on OCR, data on punched card.
	Communication Link	It represents the data received or to be transmitted from an external system.
	Magnetic Disk	It represent data input or output from and to a magnetic disk.

Ans.(d) Primary memory v/s secondary memory :
Refer to Q.2.

Q.25 Draw a block diagram of basic architecture of a computer system. [R.T.U. 2014]

OR

Design and explain the architecture of computer. [R.T.U. 2011]

Ans. Refer to Q.11.

Q.26 What is the function of CPU in a computer system? [R.T.U. 2010, 2007]

Ans. The Central Processing Unit (CPU) : The CPU is the brain of a computer system. All major calculations and comparisons performed by a computer are carried out inside its CPU. The CPU is also responsible for activating and controlling the operations of the computer system. Hence, no other single component of a computer determines its overall performance, as much as the CPU. In order to be able to quickly evaluate any computer's capabilities, it is important to know how CPUs are internally structured, how different CPUs differ from each other, and how CPU speed is evaluated. These and other related concepts about CPU are described below :

(i) The Control Unit : The two basic component of a CPU are the control unit and the arithmetic logic unit. The control unit of the CPU selects and interprets program instructions, and then sees that they are executed.

It has some special purpose registers, and a decoder to perform these activities. The special purpose registers, namely the instruction register and the program control register, respectively hold the current instruction and the next instruction to be executed, and in this way help the control unit in instruction selection. On the other hand, the decoder has the necessary circuitry to decode and interpret the meaning of every instruction supported by the CPU. Each instruction is accomplished by microcode i.e. very basic directions, which tell the CPU how to execute the instruction.

Although, the control unit does not perform any actual processing of the data, it acts as a central nervous system for the other component of the computer it manages and coordinates the entire computer system, including the input and output units. It obtains instructions

from the program stored in the main memory, interprets the instructions, and issues signals, which cause other units of the system to execute them.

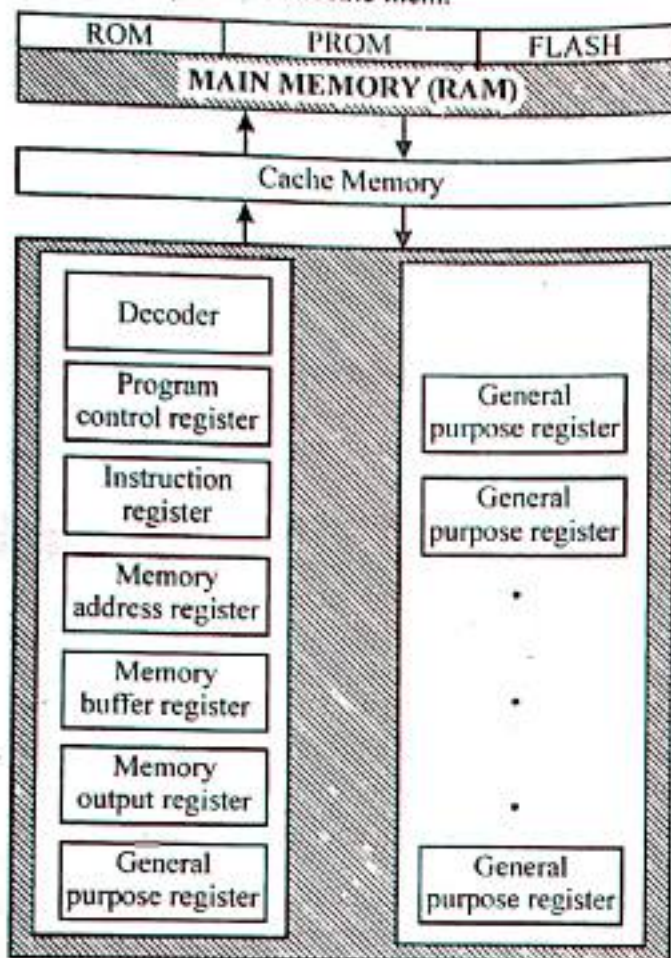


Fig. : Processor and memory architecture of a computer system

(ii) The Arithmetic Logic Unit (ALU) : The ALU of the CPU is the place, where the actual execution of the instructions takes place, during the data processing operation. That is, when the control unit encounters an instruction, which involves an arithmetic operation (such as, add, subtract, multiply, divide), or a logic operation (such as less than, equal to, greater than), it passes control to the ALU. The ALU has some special purpose registers and the necessary circuitry, to carry out all the arithmetic and logic operations, which are included in the instructions supported by the CPU. For example, the control unit might load two numbers into the registers in the ALU. Then, it might tell the ALU to add the two numbers (an arithmetic operation), or to check if the two numbers are equal (a logical operation).

In case of a microcomputer, the entire CPU (both the control unit and the ALU) is contained on a single tiny silicon chip, called a microprocessor.

Instruction Set : Every CPU has built-in ability to execute a set of machine instructions, called its instruction set. Most CPUs have 200 or more instructions (such as add, subtract, and compare) in their instruction set. The machine language designed for a processor (CPU), is based on the list of instructions supported by the CPU in its instruction set. Since each processor has a unique instruction set, machine language programs written for one computer will generally not run on another computer, with a different CPU.

CPUs made by different manufacturers have different instruction sets. In fact, different CPU models of the same manufacturer also often have different instruction sets. However, manufacturers tend to group their CPUs into "families", which have similar instruction sets. When a new CPU is developed, it is ensured that its instruction set included all the instructions in the instruction set of its predecessor CPU, plus some new ones. This manufacturing strategy is known as upward compatibility, and the new CPU is said to be upward compatible with its predecessor. This feature allows software written for a computer with a particular CPU, to work on computers with newer processor of the same family. In turn, it allows the users of these computer systems to easily upgrade their system, without worrying about converting all their existing software.

Registers : As the instructions are interpreted and executed by the CPU, there is a movement of information between the various units of the computer system. In order to handle this process satisfactorily, and to speed up the rate of information transfer, the computer uses a number of special memory units, called registers. These registers are used to hold information on a temporary basis, and are part of the CPU (not main memory).

The length of a register equals the number of bits it can store. Hence, a register that can store 8 bits is normally referred to as an 8-bit register. Most CPUs sold today, have 32-bit or 64-bit registers. The size of the registers is sometimes called the word size. The bigger the word size, the faster the computer can process a set of data. With all other parameters being same, a CPU with 32-bit registers, can process data twice as fast as one with 16-bit registers.

Although, the number of registers varies from computer to computer, there are some registers, which are common to all computers. The functions of these registers are described below.

1. Memory Address Register (MAR) : It holds the address of the active memory location. It is loaded

from the program control register, when an instruction is read from memory.

2. Memory Buffer Register (MBR) : It holds the contents of the memory word read from, or written in memory. An instruction word placed in this register is transferred to the instruction register. A data word placed in this register is accessible for operation with the accumulator register, or for transfer to the I/O register. A word to be stored in a memory location must first be transferred to the MBR, from where it is written in memory.

3. Program Control Register (PCR) : It holds the address of the next instruction to be executed. Normally, the instructions of a program are stored in consecutive memory locations, and executed in sequence, unless a branch instruction is encountered. A branch instruction is an operation, which, calls for a transfer to a nonconsecutive instruction. The address part of a branch instruction is transferred to the PC register, to become the address of the next instruction.

4. Accumulator Register (AR) : It holds the data to be operated upon, the intermediate results and the results of processing. It is used during the execution of most instructions. The results of arithmetic operations are returned to the accumulator register, for transfer to main memory, through the memory buffer register. In many computers, there are more than one accumulator registers.

5. Instruction Register (IR) : It holds the current instruction, which is being executed, as soon as the instruction is stored in this register, the operation part and the address part of the instruction are separated. The address part of the instruction is sent to the MAR, while its operation part is sent to the control unit, where it is decoded and interpreted, and ultimately command signals are generated to carry out the task specified by the instruction.

6. Input/Output Register (I/O) : It is used to communicate with the input/output devices. All input information, such as instructions and data, are transferred to this register by an input device. Similarly, all output information, to be transferred to an output device, are found in this register.

Table : Functions of various registers

S.No.	Name of register	Function
1.	Memory Address (MAR)	Holds the address of the active memory location.
2.	Memory Buffer (MBR)	Holds information on its way to and from memory
3.	Program Control (PC)	Holds the address of the next instruction to be executed
4.	Accumulator Register (AR)	Accumulator results and data to be operated upon
5.	Instruction Register (IR)	Holds an instruction, which it is being executed
6.	Input/Output (I/O)	Communicate with the I/O device

The execution of an instruction by the CPU, during program execution, normally involves the following steps:

1. The control unit takes the address of the next program instruction to be executed from the program control register, and reads the instruction from the corresponding memory address, into the instruction register of the control unit.
2. The control unit then sends the operation part and the address part of the instruction, to the decoder and the memory address register, respectively.
3. The decoder interprets the instruction, and accordingly the control unit sends signals to the appropriate unit, which needs to be involved in carrying out the task specified in the instruction. For example, if it is an arithmetic or logic operation, the signal is sent to the ALU. In this case, the control unit also ensures that the data corresponding to the address part of the instruction is loaded in a suitable register in the ALU, before the signal is sent to the ALU. The ALU performs the necessary operation on the data, and signals the control unit as soon as it has finished.
4. As each instruction is executed, the address of the next instruction to be executed is automatically loaded into the program control register, and steps 1 to 4 are repeated.

□□□

NUMBER SYSTEM

2

PREVIOUS YEARS QUESTIONS

PART A

Q.1 Convert $(651.24)_8 = (?)_2$ [R.T.U. 2016]

Ans. $(651.24)_8 = (?)_2$

6	5	1	.	2	4
↓	↓	↓		↓	↓
110	101	001	.	010	100

The octal equivalent 3-bit binary number is :

$(651.24)_8 = (110101001.010100)_2$ Ans.

Q.2 $(10110001101)_2 = (?)_{10}$ [R.T.U. 2016]

Ans. $(10110001101)_2 = (?)_{10}$

$$\Rightarrow (2^{10} \times 1 + 2^9 \times 0 + 2^8 \times 1 + 2^7 \times 1 + 2^6 \times 0 + 2^5 \times 0 + 2^4 \times 0 + 2^3 \times 1 + 2^2 \times 1 + 2^1 \times 0 + 2^0 \times 1)_{10}$$

$$\Rightarrow (1024 + 0 + 256 + 128 + 0 + 0 + 0 + 8 + 4 + 0 + 1)_{10}$$

$$\Rightarrow (1421)_{10}$$
 Ans.

Q.3 $(9896)_{10} = (?)_{16}$ [R.T.U. 2016]

Ans. $(9896)_{10} = (?)_{16}$

16	9896	8
16	618	10
16	38	6
	2	2
	0	

$$= (2 \ 6 \ 10 \ 8)_{16} = (26A8)_{16}$$
 Ans.

Q.4 $(5676)_{10} = (?)_8$ [R.T.U. 2016]

Ans. $(5676)_{10} = (?)_8$

8	5676	4
8	709	5
8	88	0
8	11	3
	1	1
	0	

$$(5676)_{10} = (13054)_8$$
 Ans.

PART B

Q.5 Solve the following :

- Convert 253.64 from base 10 to base 8
- The given hexadecimal number (1E.53) base 16 is equivalent to () base 8
- Convert the binary number (01011.1011) into decimal

- (d) Find r 's and $(r-1)$'s complement of 5308
 (e) Solve the following $(10110.01)_2 - (11010.10)_2$ using $(r-1)$'s complement.

[R.T.U. 2019]

Ans.(c) $(253.64)_{10} \rightarrow (?)_8$

8	253	5
8	31	7
8	3	
	0	

$(375)_8$

Now convert fractional part

$$.64 \times 8 = 5.12 \Rightarrow 5$$

$$.12 \times 8 = .96 \Rightarrow 0$$

$$.96 \times 8 = 7.68 \Rightarrow 7$$

$$.68 \times 8 = 5.44 \Rightarrow 5$$

$$.44 \times 8 = 3.52 \Rightarrow 3$$

$$.52 \times 8 = 4.16 \Rightarrow 4$$

$$.16 \times 8 = 1.28 \Rightarrow 1$$

$$.28 \times 8 = \text{and so on}$$

$(375.5075341)_8$

Ans.(b) $(1E.53)_{16} \rightarrow (?)_8$

$$1 \rightarrow 0001$$

$$E \rightarrow 1110$$

$$5 \rightarrow 0101$$

$$3 \rightarrow 0011$$

$(1E.53)$

0001	1110	0101	0011	0
3	6	2	4	6

$(36.246)_8$

Ans.(c) $(01011.1011)_2 \rightarrow (?)_{10}$

$$0 \ 1 \ 0 \ 1 \ 1 \ . \ 1 \ 0 \ 1 \ 1$$

$$2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0 \quad 2^{-1} \ 2^{-2} \ 2^{-3} \ 2^{-4}$$

$$0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \cdot \frac{1}{2} + \frac{1}{8} + \frac{1}{16}$$

$$= 8 + 2 + 1 \cdot (.5 + .125 + 0.625)$$

$$= (11.6875)_{10}$$

Ans.(d) r 's complement of $(5308)_{10}$

$$r^n - N$$

$$r = 10$$

$$n = 4$$

$$N = (5308)_{10}$$

$$= (10^4)_{10} - (5308)_{10}$$

$$= (10000)_{10} - (5308)_{10}$$

$$= (4692)_{10}$$

$$r - 1 \text{'s complement of } (5308)_{10}$$

$$r^n - 1 - N$$

$$r = 10$$

$$n = 4$$

$$N = (5308)_{10}$$

$$= (10^4 - 1)_{10} - (5308)_{10}$$

$$= (9999)_{10} - (5308)_{10}$$

$$= (4691)_{10}$$

Ans.(e) $(10110.01)_2 - (11010.10)_2$

$$r = 2$$

$$(r - 1) \text{ complement}$$

$$\therefore 1 \text{'s complement of } 11010.10 \text{ is } 00101.01$$

$$\text{Now, Add,}$$

$$\begin{array}{r} 10110.01 \\ + 00101.01 \\ \hline 11011.10 \end{array}$$

Since there is no carry, therefore the result is a negative and it is negative of the 1's complement of (11011.10) .

Hence the required difference is (-00100.01) i.e. (-100.01)

Q.6 Perform the following -

[R.T.U. 2019]

(a) $(9387)_{10} = ?_{(16)}$

(b) $(A15C)_{16} = ?_{(2)}$

(c) $(10101)_2 - (01110)_2$

(d) $(651.24)_{(8)} = ?_{(10)}$

(e) $(111101011)_2 = ?_{(8)}$

Ans.(a) $(9387)_{10} = (?)_{16}$

16	9387	11
16	586	10
16	36	4
16	2	2
	0	

$$(2 \ 4 \ 10 \ 11)_{16} = (24AB)_{16}$$

Ans. (b) $(A15C)_{16} = (?)_2$

$$\begin{array}{cccc} \text{A} & 1 & 5 & C \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1010 & 0001 & 0101 & 1100 = (1010000101011100)_2 \end{array}$$

Ans. (c) $(10101)_2 - (01110)_2$

$$\begin{array}{r} 10101 \\ - 01110 \\ \hline 00111 \end{array}$$

$= (00111)_2$

Ans. (d) $(651.24)_8 = (?)_{10}$

$$\begin{aligned} &= 6 \times 8^2 + 5 \times 8^1 + 1 \times 8^0 + 2 \times 8^{-1} + 4 \times 8^{-2} \\ &= 384 + 40 + 1 + 0.25 + 0.0625 \\ &= (425.3125)_{10} \end{aligned}$$

Ans. (e) $(111101011)_2 = (?)_8$

$$\begin{array}{ccc} \boxed{11} & \boxed{101} & \boxed{011} \\ \downarrow & \downarrow & \downarrow \\ 7 & 5 & 3 = (753)_8 \end{array}$$

Q.7 Perform the following :

(a) $(1101.101)_2 = (?)_{10}$

(b) $(743.91)_{10} = (?)_8$

(c) $(A1B2.6C)_{16} = (?)_{10}$

(d) $(10111010)_2 + (10101)_2 = (?)_2$

(e) $(1011011)_2 - (110101)_2 = (?)_2$ [R.T.U. 2018]

Ans. (a) $(1101.101)_2 = (?)_{10}$

$$\begin{array}{ccccccc} 1 & 1 & 0 & 1 & . & 1 & 0 & 1 \\ \uparrow & \uparrow & \uparrow & \uparrow & & \uparrow & \uparrow & \uparrow \\ 2^3 & 2^2 & 2^1 & 2^0 & & 2^{-1} & 2^{-2} & 2^{-3} \end{array}$$

$$= (1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) + (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3})$$

$$= (8 + 4 + 0 + 1) + (0.50 + 0 + 0.125)$$

$$= (13.625)_{10}$$

Ans. (b) $(743.91)_{10} = (?)_8$

Converting decimal to octal

Step 1 : Integer part = 743

8	743	7
8	92	4
8	11	3
	1	

$$(743)_{10} = (1347)_8$$

Step 2 : Fractional part = 0.91

	Fraction	Integer
$0.91 \times 8 = 7.28$	0.28	7
$0.28 \times 8 = 2.24$	0.24	2
$0.24 \times 8 = 1.92$	0.92	1
$0.92 \times 8 = 7.36$	0.36	7
$0.36 \times 8 = 2.88$	0.88	2
$0.88 \times 8 = 7.04$	0.04	7

$$(743.91)_{10} = (1347.721727)_8$$

Ans. (c) $(A1B2.6C)_{16} = (?)_{10}$

A1B2.6C

$$\begin{aligned} &= (10 \times 16^3) + (1 \times 16^2) + (11 \times 16^1) + (2 \times 16^0) \\ &\quad + (6 \times 16^{-1}) + (12 \times 16^{-2}) \\ &= (41394.421875)_{10} \end{aligned}$$

Ans. (d) $(10111010)_2 + (10101)_2 = (?)_2$

In binary addition, remember the following rules:

$$0 + 0 = 0$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

Now

$$10111010$$

$$+ 10101$$

$$\hline 11001111$$

Hence $(11001111)_2$

Ans. (e) $(1011011)_2 - (110101)_2 = (?)_2$

Number 1 in the decimal system

$$1011011_2 = 91_{10}$$

Number 2 in the decimal system

$$110101_2 = 53_{10}$$

Rules

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1 \text{ (with a borrow of 1)}$$

Their difference is

$$91 - 53 = 38$$

Result in binary form

$$38_{10} = 1001110_2$$

Q.8 Perform the following :

(i) $(89D)_{16} + (259)_{10}$

(ii) $(263)_8 + (100100101)_2$

(iii) $(10101)_2 - (01110)_2$

(iv) Multiply $(101)_2$ with $(011)_2$ [R.T.U. 2015]

Ans. (i) $(89D)_{16} + (259)_{10}$

First we convert $(89D)_{16}$ to Decimal as :

$$(89D)_{16} = 8 \times 16^2 + 9 \times 16 + D \times 16^0$$

$$= 8 \times 256 + 144 + 13 \times 1$$

$$= (2205)_{10}$$

Now adding $(2205)_{10} + (259)_{10}$

$$\begin{array}{r} 2205 \\ + 259 \\ \hline 2464 \end{array}$$

$= (2464)_{10}$

Ans. (ii) $(263)_8 + (100100101)_2$

First we convert $(263)_8$ to binary number as :

$$(263)_8 = (010110011)_2$$

Now adding $(010110011)_2 + (100100101)_2$

$$\begin{array}{r} 010110011 \\ + 100100101 \\ \hline 111011000 \end{array}$$

$= (111011000)_2$

Ans. (iii) $(10101)_2 - (01110)_2$

$$\begin{array}{r} 10101 \\ - 01110 \\ \hline 00111 \end{array}$$

$= (00111)_2$

Ans. (iv) $(101)_2 \times (011)_2$

$$\begin{array}{r} 011 \\ \times 101 \\ \hline 011 \\ 000 \\ + 011 \\ \hline 01111 \end{array}$$

Here $(101)_2 \times (011)_2 = (01111)_2$

Q.9 Convert the following :

(i) $3987_{(10)} = ?_{(16)}$

(ii) $4A5B_{(16)} = ?_{(10)}$

[R.T.U. 2013]

Ans. (i) $3987_{(10)} = ?_{(16)}$

2	3987	1
2	1993	1
2	996	0
2	498	0
2	249	1
2	124	0
2	62	0
2	31	1
2	15	1
2	7	1
2	3	1
	1	

$$(3987)_{10} = \left(\frac{1111}{16} \frac{1001}{16} \frac{0011}{16} \right)_2$$

$$= (F93)_{16}$$

Ans. (ii) $4A5B_{(16)} = ?_{(10)}$

$$\Rightarrow 4 \times 16^3 + A \times 16^2 + 5 \times 16 + B$$

$$\Rightarrow 4 \times 16^3 + 10 \times 16^2 + 5 \times 16 + 11 = 19035$$

$$(4A5B)_{16} = (19035)_{10}$$

Q.10(a) Write r 's complement of following number where r is radix (base) of these numbers :

(i) $(10111010)_2$

(ii) $(34056)_8$

(iii) $(8750)_{10}$

(iv) $(7F3B)_{16}$

[R.T.U. 2013]

(b) Write binary codes for digits 0 to 9 in following coding systems :

(i) BCD 8421

(ii) 2421 Codes

(iii) Excess-3 Codes

(iv) Gray Codes.

[R.T.U. 2012]

Ans. (a) The r 's complement of a positive number N is defined as $(r^n) - N$

Where r is the base of the number system, n is the number of digits in the integer part of N .

(i) $(10111010)_2$

$$r^n - N$$

$$r = 2$$

$$n = 8$$

$$N = (10111010)_2$$

$$= (2^8)_{10} - (10111010)_2$$

$$= (256)_{10} - (10111010)_2$$

$$= (100000000)_2 - (10111010)_2$$

$$= (1000110)_2$$

Thus, 2's complement of $(10111010)_2$ is $(1000110)_2$

(ii) $(34056)_8$

$$r^n - N$$

$$r = 8$$

$$n = 5$$

$$N = (34056)_8$$

$$= (8^5)_{10} - (34056)_8$$

$$= (32768)_{10} - (34056)_8$$

$$= (100000)_8 - (34056)_8$$

$$= (43722)_8$$

Thus, 8's complement of $(34056)_8$ is $(43722)_8$

(iii) $(8750)_{10}$

$$r^n - N$$

$$r = 10$$

$$n = 4$$

$$N = (8750)_{10}$$

$$= (10^4)_{10} - (8750)_{10}$$

$$= 10000 - 8750$$

$$= (1250)_{10}$$

Thus, 10's complement of $(8750)_{10}$ is $(1250)_{10}$.

(iv) $(7F3B)_{16}$

$$r^n - N$$

$$r = 16$$

$$n = 4$$

$$N = (7F3B)_{16}$$

$$= (16^4)_{10} - (7F3B)_{16}$$

$$= (65536)_{10} - (7F3B)_{16}$$

$$= (10000)_{16} - (7F3B)_{16}$$

$$= (A000)_{16} - (7F3B)_{16}$$

$$= (753B)_{16}$$

Ans.(b) BCD : (Binary Coded Decimal) : This code is basically decimal number coded in binary. This is weighted code in which a 'weight' has been assigned to each symbol position in the code word. In this a string of binary digits represent a decimal digit. Each decimal digit 0 through 9 is represented by a string of four bits. We have number of BCD codes out of which 8421 is most popular.

(i) **8421 BCD Code :** The BCD (Binary Coded Decimal) is a straight assignment of the binary equivalent. It is possible to assign weights to the binary bits according to their positions. In 8421 BCD code the weights are 8, 4, 2 and 1.

For example : The binary code 1001 can be used to represent the decimal digit 9 as

$$1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 9.$$

We can have the following 8421 BCD codes for the ten decimal digits :

Table

Decimal Digit	8421 BCD Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

(ii) **2421 BCD Code :** In this code the weights are 2, 4, 2 and 1. The decimal numbers are represented in 4 bit form and the total four bit weight is $9(2 + 4 + 2 + 1)$. Thus, the 2421 BCD code represented by the decimal

numbers from 0 to 9. The following table represents the 2421 BCD codes for the ten decimal digits :

Table

Decimal Digit	2421 BCD Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

For example :

$$\begin{array}{ccc} 5 & 9 & 7 \rightarrow \text{Decimal} \\ \downarrow & \downarrow & \downarrow \rightarrow \text{8421 BCD} \end{array}$$

$$0101 \ 1001 \ 0111$$

$$\therefore (597)_{10} = (0101 \ 1001 \ 0111)_{8421 \text{ BCD}}$$

(iii) **Excess -3 Code :** Excess -3 is a non weighted code used to represent decimal numbers. This code is called excess -3 because each binary code is the corresponding 8421 code plus 3(0011).

For Example : The 8421 BCD code for the digit 1 is 1000 and is 1011 in excess -3 code.

The excess -3 codes for the ten decimal digits can be shown as :

Table

Decimal Digit	Excess-3 code
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100

(iv) **Gray Code :** It is named after Frank Gray. The gray code is also called **Reflective Digital Code** which has the special property that any two subsequent codes differ by only one bit. This is also called **Unit Distance Code**.

Conversion of a Binary Number to Gray Code

Step 1 : The first bit (MSB) of the gray code is the same as the first bit of the binary number.

Step 2 : The second bit of the gray code equals the exclusive -OR (XOR) of the first and second bits of the binary number.

Step 3 : The third bit of the gray code equal the XOR of the second and third bits of the binary number, and so on.

The exclusive OR(XOR) operation truth table is :

Table

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Q.11(a) Calculate r 's and $(r - 1)$ complement of $(42)_{10}$

(b) Convert $(1000)_2$ to Gray Code.

(c) Represent the following numbers in sign magnitude, 1's complement and 2's complement representation. Assume word length = 16 bits:

- (i) $(+34)_{10}$ (ii) $(-38.2)_{10}$
 (iii) $(-76.8)_{10}$ (iv) $(+45)_{10}$ [R.T.U. 2011]

Ans.(a) $r^n - N$

$$r = 10$$

$$n = 2$$

$$N = (42)_{10}$$

$$\begin{aligned} r^n - N &= (10^2)_{10} - (42)_{10} \\ &= 100 - 42 \\ &= (58)_{10} \end{aligned}$$

Thus, r 's complement of $(42)_{10}$ is $(58)_{10}$.

$(r - 1)$'s complement is calculated by $r^n - r^m - N$

where, r is the base of the number

n is the number of digits in integer part

m is the number of digits in fractional part

So, $r = 10$

$$n = 2$$

$$m = 0$$

$$r^n - 1 - N$$

$$N = (42)_{10}$$

$$\begin{aligned} r^n - N &= (10^2 - 1)_{10} - (42)_{10} \\ &= 99 - 42 \\ &= (57)_{10} \end{aligned}$$

Thus, $(r - 1)$'s complement of $(42)_{10}$ is $(57)_{10}$

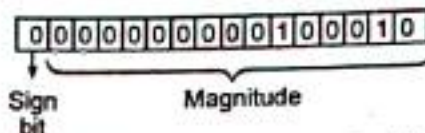
Ans.(b) $(1000)_2$

Gray code for this number will be

$$1100.$$

Ans.(c)(i) $(+34)_{10}$

$(34)_{10} = (100010)_2$ is the magnitude of the number. The '+' sign will be represented by 0. Thus the number $(+34)_{10}$ will be stored as



As the number is positive its 1's and 2's complement will be the same as sign magnitude represented.

(ii) $(-38.2)_{10}$

For $(38)_{10}$ binary is 100110

For 0.2

$$0.2 \times 2 = 0.4 - 0$$

$$0.4 \times 2 = 0.8 - 0$$

$$0.8 \times 2 = 1.6 - 1$$

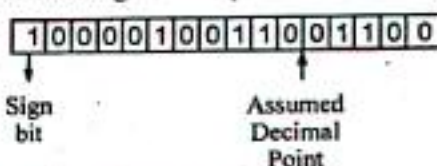
$$0.6 \times 2 = 1.2 - 1$$

$$0.2 \times 2 = 0.4 - 0$$

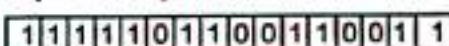
$$0.2 = 00011$$

$(38.2)_{10} = 100110.01100$

Significant magnitude representation



1's complement representation

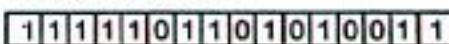


2's complement

$$11111011001.10011$$

$$+ 1.0$$

$$11111011010.10011$$



(iii) $(-76.8)_{10}$

$$2 \mid 76$$

$$2 \mid 38 \quad 0$$

$$2 \mid 19 \quad 0$$

$$2 \mid 9 \quad 1$$

$$2 \mid 4 \quad 1$$

$$2 \mid 2 \quad 0$$

$$1 \mid 0$$

$$(76)_{10} = (1001100)_2$$

$$(0.8)_{10} = 0.10011$$

$$(76.8)_{10} = (1001100.10011)_2$$

Significant magnitude representation

1 0 0 0 1 0 0 1 1 0 0 1 0 0 1 1

Sign
bit

Assumed
Decimal
Point

1's complement representation

1 1 1 1 0 1 1 0 0 1 1 0 1 1 0 0

2's complement

11110110011.10011

+ 1.0

11110110100.10011

1 1 1 1 0 1 1 0 1 0 0 0 1 1 0 0

(iv) $(+45)_{10}$

$(45)_{10} = (101101)_2$

Significant magnitude representation

Significant Magnitude Representation

0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 1

As the digit is positive, 1's complement and 2's complement will be the same.

Q.12 Write short notes on ASCII and EBCDIC codes.

[R.T.U. 2008, Raj Univ. 2001]

OR

State the difference between ASCII and EBCDIC.

[R.T.U. 2010, Raj Univ. 2002]

OR

Explain EBCDIC.

[Raj Univ. 2004]

Ans. ASCII : This code is popular in the data communication, is used almost exclusively to represent in micro computers. ASCII is of two types : ASCII-7 and ASCII-8.

ASCII-7 is seven bit code. That allows $2^7 = 128$ different characters.

ASCII-8 is a standard version of ASCII-7. It is an 8 bit code. That allows $2^8 = 256$ different characters. ASCII also uses hexadecimal as its four-to-one short cut notation for memory dump.

EBCDIC : This character code was developed by IBM. This is alphanumeric code which stands for Extended Binary Coded Decimal Interchange Code. This is broadly similar to ASCII. With the help of this code (256) different characters can be represented since this is 8 bit code. This is extended version of BCD. BCD is a 6 bit code while this is 8 bit out of which added two bits are used as additional zone bits, expanding the zone to 4 bit. It can be easily divided in 4-bit groups represented by one hexadecimal digit.

This code is used by large computers for transferring alphanumeric data. In this code we do not use a straight binary sequence for representing characters as in ASCII code. This makes the hardware requirement very simple and is related to punched card codes.

This also allows a large variety of printable characters and several nonprintable control characters.

Table

Character	EBCDIC Zone	Code Digit	Hexadecimal Equivalent
A	1100	0001	C1
B	1100	0010	C2
C	1100	0011	C3
D	1100	0100	C4
E	1100	0101	C5
F	1100	0110	C6
G	1100	0111	C7
H	1100	1000	C8
I	1100	1001	C9
J	1101	0001	D1
K	1101	0010	D2
L	1101	0011	D3
M	1101	0100	D4
N	1101	0101	D5
O	1101	0110	D6
P	1101	0111	D7
Q	1101	1000	D8
R	1101	1001	D9
S	1110	0010	E2
T	1110	0011	E3
U	1110	0100	E4
V	1110	0101	E5
W	1110	0110	E6
X	1110	0111	E7
Y	1110	1000	E8
Z	1110	1001	E9
0	1111	0000	F0
1	1111	0001	F1
2	1111	0010	F2
3	1111	0011	F3
4	1111	0100	F4
5	1111	0101	F5
6	1111	0110	F6
7	1111	0111	F7
8	1111	1000	F8
9	1111	1001	F9

EBCDIC also uses hexadecimal number system is used as shortcut notation for memory dump. This results

is a one-to-four reduction in the volume of memory dump.

Q.13 Convert the following Numbers :

(i) $(D3E0)_{16} = (?)_{10}$

(ii) $(7312)_8 = (?)_{16}$

(iii) $(736.5)_8 = (?)_{10}$

(iv) $(26.875)_{10} = (?)_2$ [R.T.U. 2010, Raj. Univ. 2004]

Ans. (i) $(D3E0)_{16} = (?)_{10}$
 $13 \times 16^3 + 3 \times 16^2 + 14 \times 16^1 + 0 \times 16^0$
 $= 53248 + 768 + 224 + 0$
 $\Rightarrow (54240)_{10}$

$\therefore (D3E0)_{16} = (54240)_{10}$ Ans.

Ans. (ii) $(7312)_8 = (?)_{16}$
 $\therefore (7312)_8 = 8^3 \times 7 + 8^2 \times 3 + 8^1 \times 1 + 8^0 \times 2$
 $\Rightarrow 3584 + 192 + 8 + 2$
 $\Rightarrow (3786)_{10}$

16	3786	10
16	236	12
16	14	12
	0	

$\Rightarrow (0 \ 12 \ 12 \ 10)_{16}$

$\therefore (7312)_8 = (CCA)_{16}$ Ans.

Ans. (iii) $(736.5)_8 = (?)_{10}$
 $\Rightarrow 7 \times 8^2 + 3 \times 8^1 + 6 \times 8^0 + (0.5 \times 8^{-1})$
 $\Rightarrow 448 + 24 + 6 + 0.0625$

$\therefore (736.5)_8 = (478.0625)_{10}$ Ans.

Ans. (iv) $(26.875)_{10} = (?)_2$

2	26	0
2	13	1
2	6	0
2	3	1
2	1	1
	0	

$\Rightarrow (011010)_2$ and $(0.875)_{10} \Rightarrow 0.875$

$0.875 \times 2 = 1.75$

$0.75 \times 2 = 1.5$

$0.5 \times 2 = 1.0 \Rightarrow (0.111)_2$

$\therefore (26.875)_{10} = (011010.111)_2$ Ans.

(b) Perform the following :

(i) $101001 - 010111$

(ii) $(738)_8 - (677)_8$

(iii) $(101)_2 + (73)_8$

(iv) $(110111)_2 + (AF)_{16}$

[R.T.U. 2010, Raj. Univ. 2002]

Ans. (a) (i) $(101011)_2 = (?)_8$

First, we will convert this into decimal equivalent, as follows:

$(101011)_2 = (2^5 \times 1 + 0 \times 2^4 + 2^3 \times 1 + 2^2 \times 0 + 2^1 \times 1 + 2^0 \times 1)_8$

or $(101011)_2 = (43)_{10}$

Now, we will convert this into octal, as follows :

8	43	3
	5	5
	0	

$= (53)_8$

$\therefore (101011)_2 = (53)_8$

(ii) $(703.5)_{10} = (?)_{16}$

$= (2 \ 11 \ 15)_{16} = (2BF)_{16}$

$V_r \times B$	$\text{Int}(V_r \times B)$
0.5×16	8.0

Thus, $(703.5)_{10} = (2BF.8)_{16}$ Ans.

(iii) $(89AE)_{16} = (?)_{10}$

or $(89AE)_{16} = (8 \times 16^3 + 9 \times 16^2 + 10 \times 16^1 + 14 \times 16^0)_{10}$

$\therefore (89AE)_{16} = (32768 + 2304 + 160 + 14)_{10} = (35246)_{10}$

(iv) $(845.5)_{10} = (?)_{16}$

Convert this into hexadecimal form as follows :

16	845	13
16	52	4
	3	3
	0	

$= (3 \ 4 \ 13)_{16} = (34D)_{16}$

$V_r \times B$	$\text{Int}(V_r \times B)$
0.5×16	8.0

$\therefore (845.5)_{10} = (34D.8)_{16}$ Ans.

Ans. (b) (i) $101001 - 010111$

1	0	1	0	0	1
-	0	1	0	1	1
	1	0	0	1	0

(ii) $(738)_8 - (677)_8$

$(738)_8$ is not valid octal number as we can have 7 as the largest digit in it. Assuming the number to be $(737)_8$

$$\begin{array}{r} 737 \\ - 677 \\ \hline 40 \end{array}$$

$$(iii) (101)_2 + (73)_8$$

First we convert $(101)_2$ to octal and get $(5)_8$

$$\text{Now } (5)_8 + (73)_8 = (100)_8$$

$$(iv) (110111)_2 + (AF)_{16}$$

First we convert $(110111)_2$ into hexadecimal as:

$$(00110111)_2 = (37)_{16}$$

Now adding $(37)_{16}$ and $(AF)_{16}$ as:

$$\begin{array}{r} 37 \\ + AF \\ \hline E6 \end{array}$$

Ans.

Ans.

Ans.

Q.15 Multiply the binary numbers $(01101)_2$ and $(1001)_2$. [R.T.U. 2008, Raj. Univ. 2001]

Ans. 01101

× 1001

01101

00000×

00000××

01101×××

01110101

Hence, $(01101)_2 \times (1001)_2 = (1110101)_2$

Ans.

Q.16 Convert the following :

(i) $(111011)_2$ to Decimal

(ii) $(111.01)_2$ to Decimal

(iii) $(ABC)_{16}$ to Octal

(iv) $(2573)_6$ to Binary

[R.T.U. 2009]

Ans. (i) $(111011)_2$ to Decimal

$$\begin{array}{r} 543210 \\ (111011)_2 = (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) \\ + (1 \times 2^1) + (1 \times 2^0) \\ = 32 + 16 + 8 + 0 + 2 + 1 \\ = (111011) = (59)_{10} \end{array}$$

Ans.

Ans. (ii) $(111.01)_2$ to Decimal

$$\begin{array}{r} 210-1-2 \\ (111.01)_2 = (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) \\ = 4 + 2 + 1 + 0 + 0.25 \\ = (7.25)_{10} \end{array}$$

Ans.

Ans. (iii) $(ABC)_{16}$ to Octal

$(ABC)_{16}$ to Binary is $(101010111100)_2$

Now binary to octal is $(101010111100)_2 = (5274)_8$

$(ABC)_{16} = (5274)_8$

Ans. (iv) $(2573)_6$ to Binary

First we convert $(2573)_6$ to decimal as

$$\begin{array}{r} 3210 \\ (2573)_6 = (2 \times 6^3) + (5 \times 6^2) + (7 \times 6^1) + (3 \times 6^0) \\ = 432 + 180 + 42 + 3 \\ = (657)_{10} \end{array}$$

Now we convert $(657)_{10}$ to Binary as

$$\begin{array}{r|l} 2 & 657 & 1 \\ 2 & 328 & 0 \\ 2 & 164 & 0 \\ 2 & 82 & 0 \\ 2 & 41 & 1 \\ 2 & 20 & 0 \\ 2 & 10 & 0 \\ 2 & 5 & 1 \\ 2 & 2 & 0 \\ & 1 & \end{array}$$

$$= (1010010001)_2$$

$$\Rightarrow (2573)_6 = (1010010001)_2$$

Ans.

PART C

Q.17 Calculate r's and $(r-1)$'s complement of the following:

(i) $(111010101)_2$

(ii) $(2435)_{10}$

(iii) $(FA2B)_{16}$

(iv) $(731.241)_8$

(v) $(24222)_5$

(vi) $(101.001)_2$

[R.T.U.]

Ans. (i) $(111010101)_2$

The r's complement of this binary number $r^2 - N$

$$r = 2$$

$$n = 9$$

$$N = 111010101$$

So r's complement of this binary number is

$$= (2)^9 - 111010101$$

$$= (512) - 111010101$$

$$= 512 - 469$$

$$= 43$$

$$[(111010101)_2 = (469)_{10}]$$

$$[(43)_{10} = (101011)_2]$$

So that value now change to the binary form

$$= (101011)_2$$

Now find the $(r-1)$'s complement of this binary number is

$$(r^n - 1) - N$$

$$= (2^6 - 1) - N$$

$$= (512 - 1) - 469$$

$$= 511 - 469$$

$$= 42$$

Now this value changes in to binary form

$$= (101010)_2$$

Ans. (ii) $(2435)_{10}$

Now first find the r 's complement of this decimal number

$$r^n - N$$

$$r = 10$$

$$n = 4$$

$$N = 2435$$

So r 's complement of this decimal number is

$$(10)^4 - 2435$$

$$= 10000 - 2435$$

$$= (7565)_{10}$$

$(r-1)$'s complement of this decimal number

$$(r^n - 1) - N$$

$$= (10^4 - 1) - 2435$$

$$= 9999 - 2435$$

$$= (7564)_{10}$$

Ans. (iii) $(FA2B)_{16}$

The r 's complement of this hexadecimal number is so first we convert this number into hexadecimal to decimal equivalent.

$$F \quad A \quad 2 \quad B$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$15 \quad 10 \quad 2 \quad 11$$

so

$$15 \times 16^3 + 10 \times 16^2 + 2 \times 16^1 + 11 \times 16^0$$

$$= 61440 + 2560 + 32 + 11$$

$$= (64043)_{10}$$

So now find the r 's complement of this decimal number

$$r^n - N$$

$$r = 10$$

$$n = 5$$

$$N = 64043$$

So

$$(10)^5 - 64043$$

$$100000 - 64043$$

$$= (35957)_{10} = (8C75)_{16}$$

So r 's complement of the given hexadecimal number is $(8C75)_{16}$.

Now find the $(r-1)$'s complement of the given hexadecimal number is

$$(FA2B)_{16} = (64043)_{10}$$

So $(r-1)$'s complement of the equivalent decimal number is

$$(r^n - 1) - N$$

$$((10)^5 - 1) - 64043$$

$$= 99999 - 64043$$

$$= (35956)_{10} = (8C74)_{16}$$

So $(r-1)$'s complement of the given hexadecimal number is $(8C74)_{16}$.

Ans. (iv) $(731.241)_8$

First we convert this given number into equivalent binary form

$$7 \quad 3 \quad 1 \quad . \quad 1 \quad 4$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$111 \quad 011 \quad 001 \quad . \quad 010 \quad 100$$

$$(731.24)_8 = (111011001.010100)_2$$

So now find the r 's complement of this given octal number change to binary equivalent number. We know that $r \rightarrow$ base of the number. So, we find the 2's complement of the given number.

So now the decimal (radix) point is removed temporarily and we will find the 2's complement.

As we see the LSB of N is 0 and Next bit is 0 and another next LSB is 1 so these these digits remain unchanged so number is

$$1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0$$

So r 's complement of given number is

$$= (000100110.101100)_2$$

Now change this binary equivalent number in to octal form

$$(000100110.101100)_2 = (046.54)_8$$

Now find the $(r-1)$'s complement of this number is

$(111011001.010100)_2$ so $(r-1)$'s complement is each 1's change to '0' and each '0' change to '1' then $(r-1)$'s complement is

$$(0001001100.101011)_2 = (046.53)_8$$

So $(r-1)$'s complement is $(046.53)_8$

Ans. (v) $(24222)_5$

Now we find the r 's complement $8(r-1)$'s complement of this given number first we converted this number to the decimal form

$$(24222)_5 = (1812)_{10}$$

Now, we find the r 's complement of this converted decimal number is

$$(1812)_{10}$$

$$r^n - N$$

$$r = 10$$

$$n = 4$$

$$N = 1812$$

So r 's complement is

$$(10)^4 - 1812$$

$$= (8188)_{10}$$

$$= (230218)_5$$

Now find the $(r-1)$'s complement of this given number so choose the decimal number

$$(1812)_{10}$$

$$(r^n - 1) - N$$

$$= (10^4 - 1)$$

$$= 9999 - 1812$$

$$= (8187)_{10} = (230217)_5$$

Ans. (vi) $(101.001)_2$

Again we use the concept used in question (iv) so now we find the r 's of the given binary number so $r = 2$ so we find the 2's complement of it.

As we see the LSB of N is 1 so we keep remains unchanged of this bit and each bit changed '1' to '0' and '0' to 1 so r 's complement of the given number is

$$r\text{'s complement } (010.111)_2$$

Now we find the $(r-1)$'s complement of this given number is so each 1's change to '0' and each 0's change to 1's so

$$(r-1)\text{'s complement is } = (010.110)_2$$

Q.18 (a) Describe methods of representing integers and floating point numbers in computer.

[R.T.U. 2012, 2010, 2009 R.U. 2002]

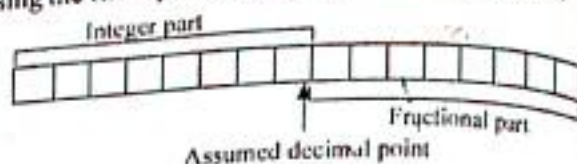
(b) Using suitable examples, explain addition and subtraction of signed integers in binary form using 2's complement representation for signed numbers.

[R.T.U. 2012]

Ans.(a) Fixed Point Representation

In the fixed point representation, as the name suggests, the position of the decimal point is fixed. The

structure of a word which has real numbers stored in it using the fixed point representation is shown below



In this, the position of the decimal point is assumed after 8 bits. Thus, the first 8 bits store the integer part of the number and the last 8 bits store the fractional part. It is not necessary that the assumed decimal point is between 8th and 9th bit in a 16 bit word. The position of the assumed decimal point can be assumed to be anywhere in the word.

Floating Point Representation

The floating point representation can be used to represent numbers in any positional number system.

Floating point representation requires that the number must be written in the format $m \cdot b^e$ or mEe . Here m is the mantissa, e is the exponent and b is the base of the number being represented using floating point notation.

The following table lists few floating point representations of numbers in various number systems:

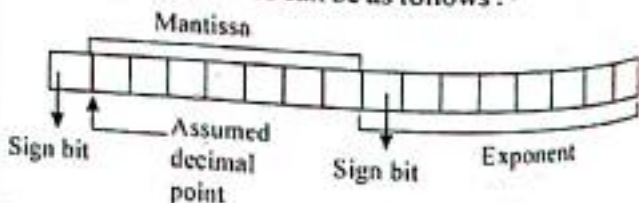
Number Exponent	Mantissa	Base	
$+2.02 \cdot 10^5$	$+2.02$	10	5
$+3.5 \cdot 10^{-3}$	$+3.5$	10	-3
$+1.011 \cdot 2^6$	$+1.011$	2	6
$-1.45 \cdot 8^5$	-1.45	8	5
$+A.F1B \cdot 16^4$	$+A.F1B$	16	4
$-2.2 \cdot 10^7$	-2.2	10	7

The mantissa is a signed fixed point number. The exponent is a signed integer which determines the actual position of the decimal point.

In this representation, mantissa and exponent have their own signs. In this section we will be talking about how reals are represented in computers, therefore we will be concentrating only on binary number system.

Once again we will have a problem of how to store the decimal point. Here also the decimal point is assumed and it is before the first bit in the mantissa part of the word.

The structure of a 16 bit word which will store floating point numbers can be as follows:



A standard floating point representation which is used is called **Normalized Representation**.

The **normalized representation** places the following restrictions on the mantissa of the number:

- The absolute value of the mantissa should be between 0 and 1 ($0 < m < 1$) i.e., it should be a fraction.
- The most significant digit after the decimal point should be non-zero for the mantissa.

To represent a real number in a word we have to follow the following steps:

Step 1: Check if the number is in the normalized form.

Step 2: If not, then normalize it. If it is in normalized form then perform Step 3.

Step 3: Write the most significant bits from left to right in the mantissa portion of the word and the exponent in the exponent part.

Step 4: Put the appropriate signs in the sign bits.

Ans. (b) One of the reasons that the 2's complement representation scheme is commonly used to store signed integers in computers is that addition and subtraction operations for signed integers are identical to the corresponding operations for unsigned integers. This means that the CPU does not need additional circuitry for unsigned addition or subtraction operations.

Let us consider some examples:

First, 0A07 and 01D3 are added. These numbers are positive whether they are interpreted as unsigned number or as 2's complement numbers. The decimal version of the addition problem is given on the right.

$$\begin{array}{r} 0A07 \quad 2567 \\ + 01D3 \quad + 467 \\ \hline 0BDA \quad 3034 \end{array}$$

The answer is correct because $BDA_{16} = 3034_{10}$.

Next 0206 and FFB0 are added. These are, of course, positive as unsigned number but when interpreted as 2's complement signed numbers, 0206 is a positive number and FFB0 is a negative number. This means that there are two decimal versions of the addition problem. The signed one is given first, then the unsigned version.

$$\begin{array}{r} 0206 \quad 518 \quad 518 \\ + FFB0 \quad + (-80) \quad + 65456 \\ \hline 101B6 \quad 438 \quad 65974 \end{array}$$

There certainly appears to be a problem because the result will not even fit in a word. In fact, because 101B6 is the hex version of 65974, there is no way to represent the correct sum of unsigned number in a word. However, if the numbers are interpreted as

signed and you ignore the extra 1 on the left, then the word 01B6 is the 2's complement representation of the decimal number 438.

Now FFE7 and FFF6 are added both negative numbers in a signed interpretation. Again, both signed and unsigned decimal interpretations are shown.

$$\begin{array}{r} FFE7 \quad (-25) \quad 65511 \\ + FFF6 \quad + (-10) \quad + 65526 \\ \hline 1FFDD \quad -35 \quad 131037 \end{array}$$

Again the sum in hex is too large to fit in two bytes, but if you throw away the extra 1, then FFDD is the correct word-length 2's complement representation of -35.

Each of the fast two additions have carry out of the usual high-order position into an extra digit. The remaining digits do not give the correct unsigned result; however, the remaining digits to give the correct 2's complement representation. The remaining digits do not always give the correct result, even for signed numbers. Consider the addition of the following two positive numbers.

$$\begin{array}{r} 483F \quad 18495 \\ + 645A \quad + 25690 \\ \hline AC99 \quad 44185 \end{array}$$

There was no carry out of the high-order digit but the signed interpretation is plainly incorrect because AC99 represents the negative number -21351. Intuitively what went wrong is that the decimal sum 44185 is bigger than the maximal value 32767 that can be stored in the two bytes of a word. However, when these numbers are interpreted as unsigned, the sum is correct.

Here is another example showing a "wrong" answer, this time resulting from adding two numbers that are negative in their signed interpretation.

$$\begin{array}{r} E9FF \quad (-5633) \quad 59903 \\ + 8CF0 \quad + (-29456) \quad + 36080 \\ \hline 176EF \quad -35089 \quad 95983 \end{array}$$

This time there is a carry but the remaining four digits 76 EF cannot be the right signed answer because they represent the positive number 30447. Again, intuition tells you that something had to go wrong because -32768 is the most negative number that can be stored in a word.

In the above example with "incorrect" signed interpretations, overflow occurred. Computer hardware can detect overflow as it performs addition and the signed sum will be correct if there is no overflow. The computer actually performs addition in binary, of course and the process is logically a right-to-left pairwise

addition of bits, very similar to the procedure that humans use for decimal addition. As the computer adds a pair of bits, sometimes there is a carry (of 1) into the next column to the left. This carry is added to the sum of these two bits and so on. The column of particular interest is the leftmost one—the sign position. There may be a carry into this position and/or a carry-out of this position into the “extra” bit. This “carry out” (into the extra bit) is what was called just “carry” above and was seen as the extra hex 1. Figure 1 identifies when overflow does or does not occur. The table can be summarized by saying that overflow occurs when the number of carries into the sign position is different from the number of carries out of the sign position.

Each of the above addition examples is now shown again, this time in binary carries are written above the two numbers.

Carry into Sign Bit?	Carry Out of Sign Bit?	Overflow?
No	No	No
No	Yes	Yes
Yes	No	Yes
Yes	Yes	No

Fig. 1: Overflow in addition.

$$\begin{array}{r}
 0000\ 1010\ 0000\ 0111\quad 0A07 \\
 + 0000\ 0001\ 1101\ 0011\quad + 01D3 \\
 \hline
 0000\ 1011\ 1101\ 1010\quad 0BDA
 \end{array}$$

This example has no carry into the sign position (bit 15) and no carry out, so there is no overflow. The carries into bits 1, 2, and 3 have no significance with respect to overflow.

$$\begin{array}{r}
 1\ 1111\ 11 \\
 0000\ 0010\ 0000\ 0110\quad 0206 \\
 + 1111\ 1111\ 1011\ 0000\quad + FFB0 \\
 \hline
 1\ 0000\ 0001\ 1011\ 0110\quad 101B6
 \end{array}$$

This example has a carry into the sign position and a carry out, so there is no overflow.

$$\begin{array}{r}
 1\ 1111\ 1111\ 11\ 11 \\
 1111\ 1111\ 1110\ 0111\quad FFE7 \\
 + 1111\ 1111\ 1111\ 0110\quad + FFF6 \\
 \hline
 1\ 1111\ 1111\ 1101\ 1101\quad 1FFDD
 \end{array}$$

Again, there is both a carry into the sign position and a carry out, so there is no overflow.

$$\begin{array}{r}
 1\ 1111\ 11 \\
 0100\ 1000\ 0011\ 1111\quad 483F \\
 + 0110\ 0100\ 0101\ 1010\quad + 645A \\
 \hline
 1010\ 1100\ 1001\ 1001\quad AC99
 \end{array}$$

Overflow does occur in this addition because there is a carry into the sign position but no carry out.

$$\begin{array}{r}
 1\ 1\ 11\ 111 \\
 1110\ 1001\ 1111\ 1111\quad E9FF \\
 + 1000\ 1100\ 1111\ 0000\quad + 8CF0 \\
 \hline
 1\ 0111\ 0110\ 1110\ 1111\quad 176EF
 \end{array}$$

There is also overflow in this addition because there is a carry out of the sign bit but no carry in.

In a computer, subtraction $a - b$ of number a and b is usually performed by taking the 2's complement of b and adding the result to a . This corresponds to adding the negative of b . For example, for the decimal subtraction $195 - 618 = -423$.

00C3

-026A

is changed to addition of FD96, the 2's complement of 026A

00C3

+ FD96

FE59

The hex digits FE59 do represent -423. Looking at the above addition in binary, you have

$$\begin{array}{r}
 11\ 11 \\
 0000\ 0000\ 1100\ 0011 \\
 + 1111\ 1101\ 1001\ 0110 \\
 \hline
 1111\ 1110\ 0101\ 1001
 \end{array}$$

Notice that there was no carry in the addition; however, this subtraction did involve borrow. A borrow occurs in the subtraction $a - b$ when b is larger than a as unsigned numbers. Computer hardware can detect a borrow in subtraction by looking at whether a carry occurred in the corresponding addition. If there is no carry in the addition, then there is a borrow in the subtraction. If there is a carry in the addition, then there is no borrow in the subtraction. (Remember that “carry” by itself means “carry out.”)

Here is one more example. Doing the decimal subtraction $985 - 411 = 574$ using word-length 2's complement representation.

03D9

-019B

is changed to addition of FE65, the 2's complement of 019B

$$\begin{array}{r}
 1\ 1111\ 1111\ 1\ 1 \\
 03D9\ 0000\ 0011\ 1101\ 1001 \\
 + FE65\ + 1111\ 1110\ 0110\ 0101 \\
 \hline
 1023E\ 1\ 0000\ 0010\ 0011\ 1110
 \end{array}$$

Discarding the extra 1, the hex digits 023E do represent 574. This addition has a carry, so there is no borrow in the corresponding subtraction.

Overflow is also defined for subtraction. When you are thinking like a person, you can detect it by the wrong answer that you will expect when you know that the difference is going to be outside of the range that can be represented in the chosen length for the representation. A computer detects overflow in subtraction by determining whether overflow occurs in the corresponding addition problem. If overflow occurs in the addition problem, then it occurs in the original subtraction problem; if it does not occur in the addition, then it does not occur in the original subtraction. There was no overflow in either of the above subtraction examples. Overflow does occur if you use word-length 2's complement representation to attempt the subtraction $-29123 - 15447$. As a human, you know that the correct answer -44570 is outside the range $-32,768$ to $+32,767$. In the computer hardware

8E3D

-3C57

is changed to addition of C3A9, the 2's complement of 3C57

$$\begin{array}{r} \text{8E3D} \quad 1000 \ 1110 \ 0011 \ 1101 \\ + \text{C3A9} \quad + \ 1100 \ 0011 \ 1010 \ 1001 \\ \hline 151E6 \quad 1 \ 0101 \ 0001 \ 1110 \ 0110 \end{array}$$

There is a carry out of the sign position but no carry in, so overflow occurs.

Although examples in this section have use word-size 2's complement representation, the same techniques apply when performing addition or subtraction with byte-size, double word-size, or other-size 2's complement numbers.

Q.19 (a) Convert following numbers as required :

Column - I	Column - II	Column - III
(i) $(27.36)_{10}$	$= ()_8$	$= ()_2$
(ii) $(27.36)_8$	$= ()_2$	$= ()_{16}$
(iii) $(110011101110101)_2$	$= ()_{16}$	$= ()_8$
(iv) $(C5D3)_{16}$	$= ()_2$	$= ()_8$

Show all steps. Use the result of column II to obtain the answer for column III.

(No weightage to be given if column III is derived directly from Column I).

(b) Add the following numbers by converting them to 2's complement :

(i) $(15)_{10}$ and $(-23)_{10}$

(ii) $(-25)_8$ and $(-32)_8$

(iii) $(23)_{16}$ and $(DC)_{16}$

(iv) $(10101011)_2$ and $(110011)_2$

[R.T.U. 2009]

Ans.(a)(i) Converting decimal to octal

Step - 1 : Integer part = 27

$$\begin{array}{r|l} 8 & 27 \\ \hline 8 & 3 \\ 0 & 3 \end{array} \quad \uparrow \quad (27)_{10} = (33)_8$$

Step - 2 : Fractional part = 0.36

	Fraction	Integer
$0.36 \times 8 = 2.88$	0.88	2
$0.88 \times 8 = 7.04$	0.04	7
$0.04 \times 8 = 0.32$	0.32	0
$0.32 \times 8 = 2.56$	0.56	2
$0.56 \times 8 = 4.48$	0.48	4
$0.48 \times 8 = 3.84$	0.84	3

$$(27.36)_{10} = (33.270243)_8$$

$$(33.270243)_8 = ()_2$$

By mapping each octal digit into its corresponding binary digit we get,

$$(33.270243)_8 = (11011.010111000010100011)_2$$

(ii) Step - 1 : $(27.36)_8 = ()_2$

By mapping each octal digit into corresponding binary digits, we get,

$$(27.36)_8 = (10111.011110)_2$$

Step - 2 : $(10111.011110)_2 = ()_{16}$

Again mapping 4 binary digits to one hex digit, we get

$$(10111.011110)_2 = (17.78)_{16}$$

(iii) Step - 1 : $(110011101110101)_2 = ()_{16}$

Mapping 4 binary digits into one hex digit, we get $(CEF5)_{16}$

Step - 2 : $(CEF5)_{16} = ()_8$

(a) $(CEF5)_{16} = ()_{10}$

$$12 \times 16^3 + 14 \times 16^2 + 15 \times 16^1 + 5 \times 16^0 = 49152 + 3584 + 240 + 5 = (52981)_{10}$$

(b) $(52981)_{10} = ()_8$

Dividing by 8

8	52981	5
8	6622	6
8	827	3
8	103	7
8	12	4
	1	

$$(52981)_{10} = (147365)_8$$

⇒ by (i) and (ii)

$$(CEF5)_{16} = (147365)_8$$

(iv) Step - 1 : $(C5D3)_{16} = ()_2$

$$(1100010111010011)_2$$

Step - 2 : $(1100010111010011)_2 = (?)_8$

$$= (142723)_8$$

Ans.(b) (i) $(15)_{10}$ and $(-23)_{10}$

Converting $(15)_{10}$ to binary

$$(15)_{10} = (1111)_2$$

Now taking 2's complement of $(23)_{10}$

$$(23)_{10} = (10111)_2$$

↓ 1's complement

$$(01000)_2$$

↓ adding 1 for 2's complement

$$(01001)_2$$

Now adding, $1111 \rightarrow$ taking 2's

$$\begin{array}{r} + 010001 \\ (111000)_2 \end{array}$$

$$\begin{array}{r} \text{complement } 00111 \\ + 1 \\ (01000)_2 \end{array}$$

$$(01000)_2 = (8)_{10}$$

Putting a -ve sign will give us the answer,

$$(-8)_{10}$$

(ii) $(-25)_8$ and $(-32)_8$

Binary equivalent of $(25)_8 = (010101)_2 \rightarrow 2$'s

$$\begin{array}{r} \text{complement } 101010 \\ + 1 \\ 101011 \end{array}$$

Binary equivalent of $(32)_8 = (011010)_2 \rightarrow 2$'s

$$\begin{array}{r} \text{complement } 100101 \\ + 1 \\ 100110 \end{array}$$

$$\begin{array}{r} \text{Adding, } 101011 \\ 100110 \\ \hline 1010001 \\ \downarrow 2's \text{ complement} \\ \text{Carry (ignore) } 101110 \\ + 1 \\ \hline 101111 \end{array}$$

Thus, $(101111)_2 = (57)_8$

Put a -ve sign to get a final answer,

i.e. $(-57)_8$

(iii) $(23)_{16}$ and $(DC)_{16}$

Binary equivalent of $(23)_{16} = (00100011)_2 \rightarrow 2$'s

$$\begin{array}{r} \text{complement } 11011100 \\ + 1 \\ \hline 11011101 \end{array}$$

Binary equivalent of $(DC)_{16} = (11011100)_2 \rightarrow 2$'s

$$\begin{array}{r} \text{complement } 00100011 \\ + 1 \\ \hline 00100100 \end{array}$$

Adding,

$$\begin{array}{r} 11011101 \\ 00100100 \\ \hline 100000001 \\ \downarrow \\ \text{Carry (ignore) } 11111110 \\ + 1 \\ \hline 11111111 \\ \hline F \quad F \end{array}$$

$= (FF)_{16}$

(iv) $(10101011)_2$ and $(110011)_2$

$(10101011)_2 \rightarrow 2$'s complement

$$\begin{array}{r} \rightarrow 01010100 \\ + 1 \\ \hline (01010101)_2 \end{array}$$

$(110011)_2 \rightarrow 2$'s complement

$$\begin{array}{r} \rightarrow 001100 \\ + 1 \\ \hline (001101)_2 \end{array}$$

$$\begin{array}{r} \text{Adding, } 01010101 \\ + 001101 \\ \hline 01100010 \\ \downarrow 2's \text{ complement} \\ 10011101 \\ + 1 \\ \hline 10011110 \end{array}$$

Ans.

Ans.

PREVIOUS YEARS QUESTIONS

PART A

Q.1 Write the format of the following functions :

- (a) *fseek* (b) *fopen*
 (c) *getch* (d) *getchar* [R.T.U. 2019]

Ans.(a) fseek() Function : *fseek()* function can be used to directly manipulate the file pointer so that it points to the desired byte offset. The prototype of *fseek()* is :

```
int fseek(FILE *fp, long offset, int origin);
```

where *fp* is a pointer to a FILE object representing a file.

```
#include <stdio.h>
```

```
int main () {
```

```
FILE *fp;
```

```
fp = fopen("file.txt", "w+");
```

```
fputs("This is Neelkanth", fp);
```

```
fseek( fp, 7, SEEK_SET );
```

```
fputs(" C Programming Language", fp);
```

```
fclose(fp);
```

```
return(0);
```

```
}
```

Ans.(b) fopen() Function : The first operation needed to be done before processing data in a file is to open the file. This is done by the *fopen()* function.

The prototype of *fopen()* is given below :

```
FILE * fopen( char * filename, char * mode);
```

Where *filename* specifies the name of the file on the secondary storage and *mode* specifies whether the file should be opened for reading or writing or for both.

The function *fopen()* returns a pointer to a FILE object if it could successfully open the file. It returns NULL if it could not. It is always good to test the value returned by *fopen()* function.

Ans.(c) getch() : *getch()* function is declared in "conio.h". This function reads a character from the console but does not echo it to the screen.

Ans.(d) getchar : *getchar()* reads a character from the console and reading is terminated when new line character (\n) is entered. The character which we enter is also displayed on the screen.

Q.2 Differentiate between Array, Structures & Unions. [R.T.U. 2019]

Ans. Difference between Array and Structure :

Basis for Comparison	Array	Structure
Basic	An array is a collection of variables of same data type.	A structure is a collection of variables or different data type.
Syntax	data type array_name[size];	Struct struct_name{ type element1; type element2; ... }variable1, variable2, ...;
Memory	Array elements are stored in contiguous memory location.	Structure elements may not be stored in a contiguous memory location.

Access	Array elements are accessed by their index number.	Structure elements are accessed by their names.
Operator	Array declaration and element accessing operator is "[]"	Structure element accessing operator is "." (Dot operator).

Difference between Structures and Unions :

Structures : Keyword – struct

The structures are the heterogeneous user defined data types where a structure content number of data types grouped together. These data types may or may not be same. E.g. to keep record of students we may use following structure

struct student

```
{
    char name [30];
    int age;
    char father_name [30];
    char class;
    int totalmarks;
}
```

Thus in this way we can group all common attributes of an object record and can access all of them via same name. E.g. we have declared variable of above structure student.

/* declaration */

struct student ram;

now if we need age of Ram we can store and retrieve it from

ram.age;

thus to access data struct we use . operator.

Union : Unions are used to save memory i.e. when we declare the union then the space allocated for use of that union is equal to memory required for the biggest variable.

Keyword union

Union is also heterogeneous data type. So in it we can use int, char, and float.

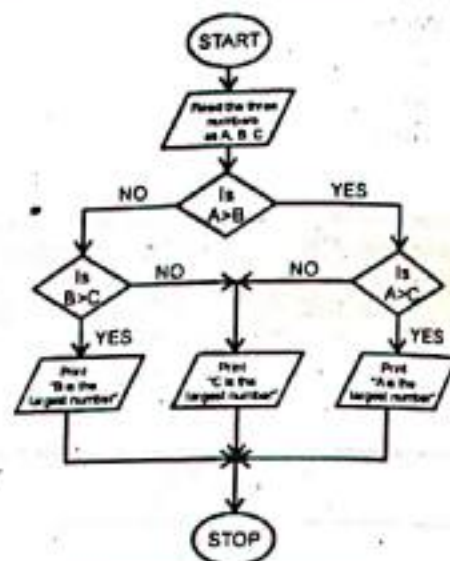
Union data

```
{
    int a;
    float p;
    char c;
} d2;
```

In union only one member can be used at a time or only one member contains valid values.

Q.3 Draw a flowchart to find the maximum among the three numbers. [R.T.U. 2019]

Ans.



Q.4 What is structure? Write syntax of structure declaration. [R.T.U. 2019]

Ans. A structure is a collection of one or more variables of different datatypes, grouped together under a single name. By using structures we can make a group of variables, arrays, pointers, etc.

Declaring a Structure : To declare a structure, we use the keyword struct. The collection of data items that need to be a part of the structure are then declared within the braces.

The syntax of declaring a structure is :

struct structure-name/tag

```
{
    type variable 1;
    type variable 2;
    :
    type variable n;
}; /* Note the semicolon after closing braces */
```

Q.5 What is type casting. [R.T.U. 2019]

Ans. Type Casting or Conversion

Type casting means of conversion of one data type to another data type. For example, a float value can be converted into an integer value. There are two types of type casting that are discussed below :

- Implicit type casting
- Explicit type casting

Programming for Problem Solving

(a) **Implicit Type Casting** : In implicit type casting, the conversion is done automatically by the compiler. In this type of conversion, a smaller data type is converted into a larger data type. For example, a float value can be converted into a double value because the size of the float value is 4 byte and the size of the double value is 8 bytes. So, it is possible for a double variable to take a float value. No operator is required in an implicit conversion. This is done without any intervention of the programmer.

(b) **Explicit Type Casting** : In explicit type casting, the conversion is carried out between two incompatible types.

For example, a float value can be converted into an integer value. This is done with the help of the (type) operator. The syntax is as follows :

data_type var1 = (type) var2

For example

int i = (int) 10.23

i.e. above example, the value (10.23) is converted into an integer value (10) and after that the resultant value (10) is assigned to the variable i. Here int is used as a (type) operator to convert the float value into an integer value.

Q.6 Write the various elements of C language (C-Tokens). [R.T.U. 2018]

Ans. Basic elements, which are used to create a C program are - the valid character set, identifiers, keywords, basic data types and their representation, constants and variables. In a C program the smallest individual units are known as C tokens. C has six types of tokens as shown in following fig. C programs are written using these tokens and the syntax of the language.

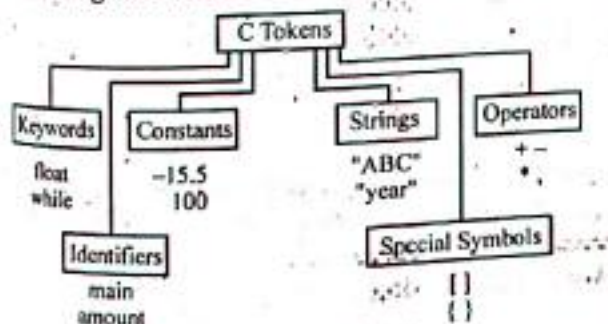


Fig. : C tokens

Q.7 Write the syntax of switch-case statement in C language. [R.T.U. 2018]

Ans.

switch (expression)

{

case constant-expression :

statement(s);

break; /*optional*/

case constant-expression :

statement(s);

break; /*optional*/

/* you can have any number of case statements*/

default : /*optional*/

statement(s);

}

Q.8 Write the difference between assignment operator and equality operator. [R.T.U. 2018]

Ans.

S. No.	Assignment Operator	Equality Operator
1.	(=) is an assignment operator in C, C++ and other programming languages. It is a binary operator which operates on two operands.	(=) is an equal to or equality operator in C and C++ only. It is a binary operator which operates on two operands.
2.	(=) assigns the value of right side expression's or variables value to the left side variable. e.g. : x = (a + b); y = x; Here, when first expression evaluator value of (a + b), it will be assigned into x and in second expression y = x; value of variable x will be assigned into y.	(=) compares value of left and side expressions, return 1 if they are equal and return 0 if not equal. e.g. : int x, y; x = 10; y = 10; if (x == y) printf("True"); else printf("False"); When expression x==y evaluates, it will return 1 (it means condition is TRUE) and "TRUE" will print.

Q.9 What is an array? Write the syntax of an array declaration in C language. [R.T.U. 2018]

Ans. Array : An array is basically a group or collection of more than one elements. As we have seen so many examples to store a single value in a variable and if we store another value in that variable then the previous is lost. So if we want to store more than one value under a single name at the same time during program execution then C provides us the facility to use arrays.

Array Declaration : Like other variables an array need to be declared so that the compiler will know what kind of an array and how large an array we want.

Syntax :

data type array name [size of array];

Example : `int num[100];`

Here, in the above example there are three parts of array declaration, `int` specifies the type of the variable, `num` specifies the name of the variable as well as name of the memory location. The `[100]` specifies the size of the array and here it is telling that we can store 100 elements of `int` type in the array `num`.

We can also declare array of other data types like :

`float salary [50];`

`char name [10];`

Q.10 Why do we include `<stdio.h>` in our programs? [R.T.U. 2018]

Ans. #include<stdio.h>

This line includes a header file which is "`stdio.h`" to our program. "`stdio.h`" means Standard Input Output header file. This file contains prototypes of some predefined functions like `printf()` and `scanf()`.

Q.11 Define a function. [R.T.U. 2018]

Ans. A function is a self-contained block of statements that performs a particular task. Once a function has been defined and packed it can be treated as a block box that takes some data from the main program and returns a value.

C functions can be defined into two categories

(a) Standard Library Functions

(b) User defined Functions

Q.12 How is an array represented in memory? [R.T.U. 2018]

Ans. Array Elements in Memory

As we have already seen that elements of array are stored continuously in memory one after one. Consider the following array declaration :

int arr[8];							
arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]	arr[6]	arr[7]
12	24	-40	11	2	0	90	77
2000	2002	2004	2006	2008	2010	2012	2014

→ Array Index
→ Values of array
→ Address of elements

Here, in the above array declaration total 16 bytes of memory is allocated, because `int` data type occupies 2 bytes in memory. All the elements of array contain garbage value because we have not initialized the array elements. Index of array is used to retrieve and store any value at a particular position of the array. Array index always starts from 0 and ends with `(size - 1)`.

Q.13 Differentiate between a character and a string. [R.T.U. 2018]

Ans. Difference between Character and String

A character is just a single character enclosed in single quotes. For example:

`char initial = 'A';` /* initial declared to be a character */

And a character string is a sequence of 0 or more characters enclosed in double quotes. Each string is terminated by a NULL byte.

`char str[] = "hello, where are you these days?";`

For example, for a string containing maximum 10 characters, we need to declare character array of size 11 bytes, as:

`char msg[11] = "hello dear";` /* 'msg' is 10 char string */

Here, 'msg' is a 10 characters string while it's declared and initialized to contain 11 characters. Last character is NULL byte to terminate the string 'msg'.

Q.14 Differentiate between `ptr++` and `*ptr++`. [R.T.U. 2018]

Ans. The difference between the `ptr++` and `*ptr++` can be explained by an example-

`*ptr` - Holds the value.

`ptr` - Holds the address.

So, if we write `ptr++`, then this will increment the address. But if we write `*ptr++` then it will increment the value of `*ptr`, because `++` operator is right associative.

For example,

if `*ptr = 5;`
if then if we print `*ptr`, then we will get the value 5
but now if we write: `*ptr++`, since it is right
associative, therefore first address will increment and
then it will print the result.

Q.15 Differentiate between a structure and an array.
[R.T.U. 2018]

Ans. Difference between Array and Structure : Refer
to Q.2.

Q.16 What do you understand by EOF? [R.T.U. 2018]

Ans. EOF : In computing, end-of-file (commonly
abbreviated EOF) is a condition in a computer operating
system where no more data can be read from a data
source. The data source is usually called a file or stream.
In the C Standard Library, the character reading functions
such as `getchar` return a value equal to the symbolic value
(macro) EOF to indicate that an end-of-file condition has
occurred. The actual value of EOF is implementation-
dependent and is distinct from all valid character codes.
Block-reading functions return the number of bytes read,
and if this is fewer than asked for, then the end of file
was reached or an error occurred.

Q.17 "Structure declaration reserves memory for the
structure" Comment on this statement with valid
justification. [R.T.U. 2018]

Ans. The statement "structure declaration reserves
memory for the structure" can be explained in following
manner. In C programming the memory is allocated only
when we declare the variables of the structure. In other
words, the memory is allocated only when we instantiate
the structure. In the absence of any variable, structure
definition is just a template that will be used to reserve
memory when a variable of type `struct` is declared. Once
the variables of a structure are defined, we can perform
a few operations on them. For example, we can use the
assignment operator (`=`) to assign the values of one
variable to another. Another important point is that
members inside the structure definition are attached to
the structure variable, they don't have any existence
without structure variable. Member names inside a
structure must be different from one another, but the
member names of two different structures can be same.

PART B

Q.18(a) Write a program in C to print inverted half
pyramid.

```
*****
****
***
**
*
```

(b) Write a program in C to check Armstrong
number of n digits. [R.T.U. 2019]

```
Ans.(a) #include <stdio.h>
#include <conio.h>
void main()
{
    int i, j, rows;
    printf("Enter the number of rows : ");
    scanf("%d", &rows);
    for(i = rows; i >= 1; --i)
    {
        for(j = 1; j <= i; ++j)
        {
            printf("* ");
        }
        printf("\n");
    }
    getch();
}
```

```
Ans.(b) #include <math.h>
#include <stdio.h>
int main()
{
    int num, originalNum, remainder, n = 0;
    float result = 0.0;
    //printf("Enter an integer : ");
    //scanf("%d", &num);
    num = 153;
    originalNum = num;
    while (originalNum != 0)
    {
        originalNum /= 10;
        ++n;
    }
```

```

    originalNum = num;
    while (originalNum != 0)
    {
        remainder = originalNum % 10;
        result += pow(remainder, n);
        originalNum /= 10;
    }
    if ((int)result == num)
        printf("%d is an Armstrong number.", num);
    else
        printf("%d is not an Armstrong number.",
num);
    return 0;
}

```

Q.19(a) What will be the output of the following "C" code?

```

#include
void main() {
    int i = -1, j = -1, k = 0, l = 2, m;
    m = i++ && j++ && k++ || l++;
    printf("%d%d%d%d", i, j, k, l, m);
}

```

[R.T.U. 2019]

(b) Explain the terms Identifiers and Data Types in C. Explain the rules of Identifier declaration in C language. List all the Data Types of C with their storage size and storage format. [R.T.U. 2019]

Ans.(a) 00131

Ans.(b) Identifier or word is a name to identify the particular information. For example, the following is a statement with the identifier and value.

```

Emp_Name = "Karthi"
Emp_Name = "Amir"
Emp_Name = "Mahesh"

```

Here Emp_Name is an identifier, common name, which is used to identify the employee names.

Rules for the Identifier or Word

- The first character must be an alphabet or underscore (_)
- Digits may be included in the variable
- The maximum number of characters in a word are 32 (It may vary depending upon the platform)
- No other special characters are allowed.

Data Types of C with their Storage Size and Storage Format :

Data Types : Data type is used to define the type of variable that we declare at the beginning of the program. There are basically two types of data types.

1. Primary Data types
2. User Defined Data types

A. Primary Data Types

The primary data types in C are follows :

1. int
2. float
3. char
4. void

The user defined data types are : structure, array, union, enum, etc. These data types are discussed later.

1. int : An integer variable can store any number in the range -32768 to + 32767. That is because an integer variable always occupies two bytes in memory and in two bytes we cannot store a number bigger than +32767 or smaller than - 32768.

Example :

```

int abc;
int i;

```

C offers a variation of the integer data types that will provide what are called long integer values. Long integers occupy double space than int, it mean long integers would occupy four bytes of memory. Long variables which hold long integers are declared using the keyword long. The values of a long integer can vary from : 2147483648 to +2147483647.

Example :

```

long int i;
long int ABC;

```

2. char : char data type can store any single character and occupy only 1 byte of memory space. Declaration of char data type is as :

```
char ch;
```

3. float : A float occupies four bytes in memory and can range from -3.4e38 to +3.4e38. If this is insufficient then C offers a double data type which occupies 8 bytes in memory and has a range from -1.7e308 to +1.7e308. A variable of type float and double can be declared as :

Example :

```

float marks;
double salary, amt;

```

If there is any requirement to store a value that can not be stored in double data type variable then we

Programming for Problem Solving

can use **long double**, which occupies 10 bytes of memory.

Example :

`long double average;`

4. **void** : **void** is a special data type which cannot store any type of constant value. We can say it a *dummy* data type.

Primary data types discussed above have also been explained in the following table with their Range, Memory Space and Format Specifiers.

Table : Data Type

Data Type	Range	Bytes	Format
signed char	-128 to + 127	1	%c
unsigned char	0 to 255	1	%c
signed int	-32768 to + 32767	2	%d
unsigned int	0 to 65535	2	%u
long signed int	-2147483648 to + 2147483647	4	%ld
long unsigned int	0 to 4294967295	4	%lu
float	-3.4e38 to + 3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to + 1.7e4932	10	%Lf

B. User Defined Data Type : Type definition is a feature in C allows users to define an identifier that would represent an existing data type. The user defined data type can later be used to declare variables.

Its general form is `typedef type identifier where` type refers to an existing data type and identifier refers to the "new name given to the data type"

e.g. `typedef int marks`

`typedef float percentage`

Here units symbolize int and marks symbolize float. They can be later used to declare variables as follows

`units batch1, batch2 and marks name1 [50], name2 [50]`

where batch1 and batch2 are declared as int variables and name 1 [50] and name 2 [50] are declared as 50 element floating point array variables.

The main advantage of the data type (typedef) is, we can create meaningful data type names for increasing the readability of the programme

These are defined data types are : arrays, structures, union, enum etc.

Structures : Refer to Q.2.

Union : Refer to Q.2.

Pointers : Pointers are the special variables used to point other storage spaces of same kind as the name indicate. Empty data set is used in functions.

Enumeration : An enumeration data type is a user defined type, with values ranging over a finite set of identifiers called enumeration constant. For example

`enum color {red, blue, green};`

This defines color to be of a new data type which can assume the value, red, blue, or green. Each of these is an enumeration constant. In the program, color can be used as a new type. A variable of type color can have any one of the three values: red, blue or green. For example, the statement

`color c;`

defines c to be of type color. Internally, the C++ compiler treats an enum type (such as color) as an integer itself. The above identifiers red, blue, and green represent the integer values of 0, 1 and 2 respectively. So, the statements

`c = blue;`

`count << "As an int, c has the value" << c;`

will print As an int, c has the value 1

Constant values can be explicitly specified for the identifiers. When the value for one identifier is specified in this manner, the value of the next element is incremented by one (next higher integer). For example, if the definition of color is

`enum color {red = 10, blue, green = 34};`

then the statement `c = red` will assign the value 10 to c. Therefore, the statement

`c = blue;`

assigns the value 34 to c. (If no value is specified for green in the declaration, it would assume the value 12).

Enumeration is a convenient way to associate constant integers with meaningful names. They have the advantage of generating the values automatically. Use of enumeration constants, in general makes the program easier to read and change at a later date.

Suitable code of 'C' language

// Initialization of an object of the enumeration type

#include <stdio.h>

enum SWITCH {off,on};

main ()

{

enum SWITCH s1 = on;

enum SWITCH s2 = s1, s3 = 0;

printf("The value of enumeration object s1 is %d\n", s1);

printf("The value of enumeration object s2 is %d\n", s2);

printf("The value of enumeration object s3 is %d\n", s3);

}

Output -

The value of enumeration object s1 is 1

The value of enumeration object s2 is 1

The value of enumeration object s3 is 0

Remarks :

- Enumerations behave like integers, but it is common for a compiler to issue a warning message when an object of an enumeration type is initialized with something other than one of its constants or an expression of its type
- When the enumeration objects are initialized with integers, the compiler will not check that whether the initialized value is valid for such an enumeration or not
- Thus, it is even possible to initialize s3 with -8, 9 or any other integer value.

Q.20 Write a program to swap two values using functions, use 2 cases [R.T.U. 2019]

(a) call by value and

(b) call by reference

Ans.(a) Swapping using Call by Value method:

```
#include<stdio.h>
#include<conio.h>
void swap(int, int);
void main()
{
    int x, y;
    printf("Enter the value of x and y\n");
    scanf("%d%d", &x, &y);
    printf("Before Swapping\nx = %d\nty = %d\n", x, y);
    swap(x, y);
    printf("After Swapping\nx = %d\nty = %d\n", x, y);
    getch();
}
void swap(int a, int b)
{
    int temp;
    temp = b;
    b = a;
    a = temp;
    printf("Values of a and b is %d %d\n", a, b);
}
```

Output:

Enter the value of x and y

Before Swapping

x = 10

y = 5

Values of a and b is 5 10

After Swapping

x = 5

y = 10

Ans. (b) Swapping using Call by Reference method:

```
#include <stdio.h>
void swap(int*, int*);
int main()
{
    int x, y;
    printf("Enter the value of x and y\n");
    scanf("%d %d", &x, &y);
    printf("Before Swapping\nx = %d\nty = %d\n", x, y);
    swap(&x, &y);
    printf("After Swapping\nx = %d\nty = %d\n", x, y);
    return 0;
}
void swap(int *a, int *b)
{
    int temp;
    temp = *b;
    *b = *a;
    *a = temp;
}
```

Output:

Enter the value of x and y

Before Swapping

x = 10

y = 5

After Swapping

x = 5

y = 10

Q.21 Explain the following conditional statements with proper syntax and example-

(a) If-else statement

(b) Switch Case

(c) While Loop

(d) Do-while Loop

[R.T.U. 2019]

Ans.(a) if - else Statement

if - else statement is used for bi-directional conditional control statements. In this statement it first checks the condition and then transfers the control in two possible directions. The direction can be true block (one or more statements) or false block (one or more statements). If the condition is true then the true block will be executed otherwise false block.

Syntax :

```
if(condition)
    statement;
else
    statement;
```

Example :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n;
    clrscr();
    printf("\nEnter a no. to be checked");
    scanf("%d",&n);
    if(n>0)
        printf("\n %d is positive",n);
    else
    if(n<0)
        printf("\n %d is negative",n);
    else
        printf("\n %d is zero",n);
}
```

Ans.(b) Switch Case

The switch statement allows us to make a decision from a list of case values and execute the case which is matched to the condition given. The general syntax of switch statement is as follows:

Syntax :

```
switch ( integer expression)
{
    case constant 1:
        statement(s);
        break;
    case constant 2:
        statement(s);
        break;
    case constant 3:
        statement(s);
        break;
```

default:

statement(s);

Example :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char ch;
    clrscr();
    printf("\nEnter a character");
    scanf("%c",&ch);
    switch(ch)
    {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            printf("\nThe character is vowel");
            break;
        default:
            printf("\nThe character is not vowel");
    }
}
```

Ans.(c) while Loop

while loop is an entry control or top tested loop. It means the test condition is evaluated first and if the condition is true first time then the body of the loop is executed. After execution of body of loop control again goes to the test condition, if this time the test condition is true then body of loop is executed again and the process goes on until the test condition becomes false.

Syntax :

```
initialization;
while(test condition)
{
    statement 1;
    statement 2;
    _____
    statement n;
    increment / decrement;
}
```

Example :

```
main ()
{
```

```

int i = 1;
while ( i <= 10)
{
    printf( "%d\n", i);
    i++;
}

```

Ans.(d) Do-while Loop

do-while loop is an exit control or bottom tested loop. This loop almost works like while loop but there is a difference, that it first executes the body of the loop and then checks whether the condition is true or not. If the condition is false at the first time, then it will also execute the statements at least once.

Syntax :

```

initialization;
do
{
    statement 1;
    statement 2;
    _____
    statement n;
    increment / decrement;
} while( Test condition);

```

Example :

```

main()
{
    int a = 5 ;
    while ( a<5 )
    {
        printf("Hello World
\n");
    }
}

```

Here, the condition fails for the first time itself the `printf()` will not get executed.

```

main()
{
    int a = 5 ;
    do
    {
        printf("Hello World
\n");
    }while(a<5);
}

```

In this example, the `printf()` would be executed once, since first time body of the loop is executed and then the condition is tested.

Q.22 What do you mean by function? Also explain types of functions available in C. [R.T.U. 2019]

Ans. Functions : A function is a self-contained block of statements that performs a particular task. Once a function has been defined and packed it can be treated as a block box that takes some data from the main program and returns a value.

C Functions can be defined into two categories which are :

- (a) Standard Library Functions
- (b) User Defined Functions

(a) Standard Library Functions: Library functions are already defined in the system library of C compiler. Library functions are also called pre defined functions. To use these library function, programmer need to include a particular header file related to that library function.

For example, if we want to use any mathematical library function, such as `sqrt()`, `abs()` then "`math.h`" header file is required to include before `main()` function.

```
#include<math.h>
```

Similarly if programmer wants to perform any string operation like `strcmp()`, `strcat()` etc. then "`string.h`" header file is required.

```
#include<string.h>
```

(b) User Defined Functions: User defined functions are those whose definitions are defined by the user itself and it can be called in the `main()` function or in any other user defined function. `main()` is also a user defined function.

1. Parts of a User Defined Function

- (i) Prototype / Declaration
- (ii) Function Calling
- (iii) Function Definition

(i) Prototype / Declaration : Prototype or declaration of a function tells the compiler that this function will be used later in the program just like we declare variables before its use. It is compulsory to declare a function before its use. Declaration of a function tells the following information: -

Syntax:

```
return type <function name>(arguments data types);
```

Here,

Return type : Return type means that our function will return a value after executing the statements defined in function definition and it tells the data type of that value.

Function Name : It belongs to the name of the function by which it will be identified.

Arguments Type : When we call any function it also takes some variables or constant values. Here we declare the data types of that variables or constants.

Example : `int factorial(int);`

Here, factorial is a function name which will take an integer type value as an argument and will return an integer type value after executing the statements written in the definition.

(ii) Function Calling : To execute the definition of a function we have to call that function in the main() function block. When we call any function we can pass the arguments that can be used by function definition and after processing of statements defined in function definition, function call can also receive the value return by function definition.

Syntax:

variable name = Function name(variables/ constants);

Example :

`fact = factorial(n);`

Here, fact is an integer type variable which will receive the factorial of the value stored in variable n, here n is integer type variable and have passed to the definition of function factorial as an argument.

(iii) Function Definition : The third and last part of the function is the definition of function in which all the statements are defined and execute when we call the function.

Syntax: return type function name(arguments with their data types)

```
{
    Statement 1;
    Statement 2;
    _____;
    _____;
}
```

Example : `int factorial(int n)`

```
{
    int i, f = 1;
    for(i = n; i > 0; i--)
    {
        f = f * i;
        n--;
    }
    return f;
}
```

Here, the definition of the function factorial is defined. In this block the factorial of the value n will be calculated and it will be returned by a statement return f and assigning the value of variable f to the variable which we have used at the time of function call.

Categories of Function

(i) Function having no Return type and no Arguments

(ii) Function having no Return type but Arguments

(iii) Function with Return type but no Arguments

(iv) Functions with Return type and Arguments

(i) Function having no Return type and no Arguments

In this category we use a function which neither has return type nor arguments. It means we cannot pass any arguments to that and definition of the function can't return any value.

Example :

```
void display(); /*function prototype*/
void main()
{
    display(); /*function call*/
}
void display() /*function definition*/
{
    printf("Hello world");
}
```

(ii) Function having no Return type but Arguments

In this category we can pass arguments to the function definition but definition of function can't return any value.

Example :

```
void swap(int, int); /*function prototype*/
void main()
{
    int a=10, b=20;
    swap(a,b); /*function call*/
}
void swap(int a, int b) /*function definition*/
{
    int c;
    c=a;
    a=b;
    b=c;
    printf("a=%d,b=%d",a,b);
}
```

(iii) Function with Return type but no Arguments

In this category of function definition we can return any type of value but at the time of calling we can't pass any argument to function definition.

Example :

```
int square();           /*function prototype*/
void main()
{
    int s;
    s=square();          /*function call*/
    printf("Square=%d",s);
}
int square()             /*function definition*/
{
    int a=5;
    return a*a;
}
```

(iv) Functions with Return type and Arguments

This category of function is used mostly because in this type we can pass both arguments to function definition and definition can return any type of values.

Example :

```
int sum(int , int);
/*function prototype*/
void main()
{
    int a=10,b=20,c;
    c=sum(a,b);
    printf("Sum=%d",c);
}
int sum(int a, int b)
{
    return a+b;
}
/*function definition*/
```

Q.23 Write a program to find the given number is palindrome or not. [R.T.U. 2019]

Ans. C program for palindrome number

```
#include <stdio.h>
int main()
{
    int n, reverse = 0, t;
```

```
printf("Enter a number to check if it is a palindrome
or not \n");
scanf("%d", &n);
t = n;
while (t != 0)
{
    reverse = reverse * 10;
    reverse = reverse + t%10;
    t = t/10;
}
if (n == reverse)
    printf("%d is a palindrome number.\n", n);
else
    printf("%d isn't a palindrome number.\n", n);
return 0;
}
```

Output :

```
Enter a number to check if it is a palindrome or
not
121
121 is a palindrome number
```

Q.24 Write a program to swap two numbers using call by value and call by reference method.

[R.T.U. 2019]

Ans. Call by Value :

```
#include <stdio.h>
//Function Prototype void swapx (int x, int y);
// Main function
int main ()
{
    int a = 10, b = 20;
    // Pass by values
    swapx (a, b);
    printf ("a = %d b = %d\n", a, b);
    return 0;
}
//swap functions that swaps
//two values
void swapx (int x, int y)
{
    int t;
    t = x;
    x = y;
    y = t;
    printf ("x = %d y = %d\n", x, y);
}
```

Output :
 x = 20 y = 10
 a = 10 b = 20

Thus actual values of a and b remain unchanged even after exchanging the values of x and y.

Call by Reference :

#include <stdio.h>

//Function Prototype

void swapx (int*, int*);

// Main function

int main ()

{
 int a = 10, b = 20;

// Pass reference

swapx (&a, &b)

printf ("a = %d b = %d\n", a, b);

return 0;

}
 //functions to swap two variables

//by references

void swapx (int*x, int*y)

{
 int t;

t = *x;

*x = *y

*y = t;

printf ("x = %d y = %d\n" *x, *y);

}

Output :
 x = 20 y = 10
 a = 20 b = 10

Thus actual values of a and b get changed after exchanging the values of x and y.

Q.25 Write the programmes to print following patterns using loop :

(a) * (b) 1
 * * 0 1
 * * * 1 0 1
 * * * * 0 1 0 1 [R.T.U. 2018]

Ans.(a) Print Star Triangle

#include<stdio.h>

#include<conio.h>

void main()

{

int i,j;
 for(i=1; i<=4; i++)
 {
 for(j=1; j<=i; j++)
 {
 printf("*");
 }
 printf("\n");
 }
 getch();
 }

Ans. (b)

#include<stdio.h>

int main()

{

int i,j;

int count = 1;

for (i = 1; i <= 4; i++)

{

printf("\n");

for (j = 1; j <= i; j++)

{

printf("%d", count % 2);

count++;

}

if (i % 2 == 0)

count = 1;

else

count = 0;

}

return(0);

}

Q.26 What do you mean by identifiers? Write the rules for identifier declaration in C. [R.T.U. 2018]

Ans. Identifier : Identifiers are names for entities in a C program, such as variables, arrays, functions, structures, unions and labels. An identifier can be composed only of uppercase, lowercase letters, underscore and digits, but should start only with an alphabet or an underscore. If the identifier is not used in an external link process, then it is called as internal.

Example : Local variable.

If the identifier is used in an external link process, then it is called as external.

Example : Global variable.

An identifier is a string of alphanumeric characters that begins with an alphabetic character or an underscore character that are used to represent various programming elements such as variables, functions, arrays, structures, unions and so on. Actually, an identifier is a user defined word. There are 53 characters, to represent identifiers. They are 52 alphabetic character (i.e. both uppercase and lowercase alphabets) and the underscore character. The underscore character is considered as a letter in identifiers. The underscore character is usually used in the middle of an identifier. There are 63 alphanumeric characters, i.e. 53 alphabetic characters and 10 digit (i.e. 0-9).

Rules for identifier declaration in C :

1. The first character in an identifier must be an alphabet or an underscore and can be followed only by any number alphabets, digit or underscores.
2. They must not begin with a digit.
3. Uppercase and lowercase letters are distinct. That is, identifiers are case sensitive.
4. Commas or blank spaces are not allowed within an identifier.
5. Keywords cannot be used as an identifier.
6. Identifiers should not be of length more than 31 characters.
7. Identifiers must be meaningful, short, quickly and easily typed and read.

Q.27 Write a program in C language to determine the youngest of three boys i.e. Shyam, Ajay and Mohan, if their ages are input through the keyboard. [R.T.U. 2018]

Ans.

```
#include<stdio.h>
#include<math.h>
void main()
{
    int ajay,mohan,shyam;
    printf("enter the ages of ajay,mohan,shyam");
    scanf("%d%d%d",&ajay,&mohan,&shyam);
    if((ajay>mohan)&&(shyam>ajay))
    {
        printf("\n the youngest of the three is mohan");
    }
    if((mohan>ajay)&&(shyam>ajay))
    {
        printf("\n the youngest of the three is ajay");
    }
}
```

```
}
if((mohan>shyam)&&(ajay>shyam))
{
    printf("\n the youngest of the three is shyam");
}
}
```

Q.28 Write a program to print the table of any given input number N. [R.T.U. 2018]

Ans. Print Table of Any Number Program in C

The concept of generating table of any number N is to multiply particular number from 1 to 10. The concept of generating table is :

```
number * 1
number * 2
number * 3
number * 4
number * 5
number * 6
number * 7
number * 8
number * 9
number * 10
```

Program :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,no,table=1;
    clrscr();
    printf("Enter any number :");
    scanf("%d",&no);
    printf("Table of %d \n",no);
    for(i=1;i<=10;i++)
    {
        table=no*i;
        printf("%d",table);
        printf("\n");
    }
    getch();
}
```

Output :

```
Enter any number : 5
Table of 5
5
```

Q.29 Define a pointer. Write a program to swap two integer variables using pointer. [R.T.U. 2018]

Ans. Pointer : Pointers in C language is a variable that stores/points the address of another variable. A Pointer in C is used to allocate memory dynamically i.e. at runtime. The pointer variable might be belonging to any of the data type such as int, float, char, double, short etc.

Pointer Syntax : data_type *var_name;

Example : int *p; char *p;

Where, * is used to denote that "p" is pointer variable and not a normal variable.

Key points to remember about pointers in C :

- Normal variable stores the value where as pointer variable stores the address of the variable.
- The content of the C pointer always be a whole number i.e. address.
- Always C pointer is initialized to null, i.e. int *p = null.
- The value of null pointer is 0.
- "&" symbol is used to get the address of the variable.
- "*" symbol is used to get the value of the variable that the pointer is pointing to.
- If a pointer in C is assigned to NULL, it means it is pointing to nothing.
- Two pointers can be subtracted to know how many elements are available between these two pointers.
- But, Pointer addition, multiplication, division are not allowed.
- The size of any pointer is 2 byte (for 16 bit compiler).

Program to swap two integer variables :

```
#include<stdio.h>
#include<conio.h>
void swap(int*, int*);
void main()
```

```
    int a, b;
    clrscr();
    printf("\n enter two numbers:");
    scanf("%d%d", &a, &b);
    swap(&a, &b);
    printf("\n values after swapping are a = %d
and b = %d", a, b);
}
void swap(int* x, int* y)
{
    int t;
    t = *x;
    *x = *y;
    *y = t;
    getch();
}
```

Output : Enter two numbers :

a = 20

b = 10

Values after swapping are

a = 10

b = 20

Q.30 How are strings read from the standard input device? Explain the different functions used to perform string input operation. [R.T.U. 2018]

Ans. Using %s format specifier when we read a string, C compiler automatically assigns all the characters in separate memory blocks and add a null character ('\0') at the end of the string. At the time of printing %s format specifier is used in printf() function and it prints the character until it reaches '\0';

Example :

```
main()
{
    char name[25];
    printf("Enter Your Name:-");
    scanf("%s", &name);
    printf("\nHello %s!", name);
}
```

Here is a simple run of the program...

Enter Your Name :- SANJAY

Hello SANJAY!

Some other String Input/Output Functions

gets() : gets() function is declared in "stdio.h". Using this function we can read more than one word

(including spaces) because it gets terminate when new line character (`\n`) is encountered.

puts() : `puts()` function is also declared in "`stdio.h`". This function can print a string on console, and has inbuilt new line character (`\n`).

getch() : Refer to Q.1(c).

getche() : `getche()` function is also declared in "`conio.h`". This function reads a character from the console and echo it on the screen.

getchar() : Refer to Q.1(d).

putchar() : `putchar()` prints the character argument on the standard output.

If we are using `getch()`, `getche()`, `getchar()` or `putchar()` for reading or displaying a string we require a loop as these functions/macros are used to read or display a single character.

A program to find occurrence of any particular character in a string.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char c,str[10];
    int i,count=0;
    clrscr();
    printf("\nEnter a string: -");
    gets(str);
    printf("\nEnter a character: -");
    scanf("%c",&c);
    for(i=0;str[i]!='\0';i++)
    {
        if(str[i]==c)
            count++;
    }
    printf("\nThis character comes in string %d times",count);
}
```

Output :

```
Enter a string: - INDIA IS GREAT
Enter a character: - A
This character comes in string 2 times
```

Q.31 Write a program in C that replaces a given character with another character in the string.
[R.T.U. 2018]

```
Ans. #include <stdio.h>
#include <string.h>
int main()
```

```
{
    char str[100], ch, Newch;
    int i;
    printf("\n Please Enter any String : ");
    gets(str);
    printf("\n Please Enter the Character that you want to Search for : ");
    scanf("%c", &ch);
    getchar();
    printf("\n Please Enter the New Character : ");
    scanf("%c", &Newch);
    for(i = 0; i <= strlen(str); i++)
    {
        if(str[i] == ch)
        {
            str[i] = Newch;
        }
    }
    printf("\n The Final String after Replacing All Occurrences of '%c' with '%c' = %s", ch, Newch, str);
    return 0;
}
```

Output:

```
Please Enter any String : Neelkantg
Please Enter the Character that you want to Search for : g
Please Enter the New Character : h
The Final String after Replacing All Occurrences of 'g' with 'h' = Neelkanth
```

Q.32 Write a program to create a structure with the information given below. Then read and print the data.

(a) *Emp_Id*

(b) *Name-*

(i) *First Name*

(ii) *Middle Name*

(iii) *Last Name*

(c) *Address-*

(i) *Area*

(ii) *City*

(iii) *State*

(d) *Age*

(e) *Salary*

(f) *Designation*

[R.T.U. 2018]

```

Ans. #include <stdio.h>
/*structure declaration*/
struct employee{
    int empId;
    char firstname[30];
    char middlename[30];
    char lastname[30];
    char area[50];
    char city[30];
    char state[30];
    float age;
    float salary;
    char designation[30];
};

int main()
{
    /*declare structure variable*/
    struct employee emp;

    /*read employee details*/
    printf("\nEnter details :\n");
    printf("Emp_Id :");
    scanf("%d", &emp.empId);
    printf("First Name :");
    gets(emp.firstname);
    printf("Middle Name :");
    gets(emp.middlename);
    printf("Last Name :");
    gets(emp.lastname);

    printf("Address :\n");
    printf("Area :");
    gets(emp.area);
    printf("City :");
    gets(emp.city);
    printf("State :");
    gets(emp.state);

    printf("Age :");
    scanf("%f", &emp.age);
    printf("Salary :");
    scanf("%f", &emp.salary);
    printf("Designation :");
    gets(emp.designation);
}

```

```

/*print employee details*/
printf("\nEnter detail is:\n");
printf("Emp_Id: %d\n", emp.empId);

printf("Name-\n");
printf("First Name: %s\n", emp.firstname);
printf("Middle Name: %s\n", emp.middlename);
printf("Last Name: %s\n", emp.lastname);

printf("Address-\n");
printf("Area: %s\n", emp.area);
printf("City: %s\n", emp.city);
printf("State: %s\n", emp.state);

printf("Age: %f\n", emp.age);
printf("Salary: %f\n", emp.salary);
printf("Designation: %s", emp.designation);
return 0;
}

```

Q.33 What is the advantage of using structures?
Differentiate between a structure and a union.
[R.T.U. 2018]

Ans. Advantages of Structure :

1. Structure is collection of different types of variables referenced under one name. By this we need not define same category variable types in different variables.
2. Memory is not allocated at the time of declaration of structure. After creation of structure variables memory is allocated.
3. Nested structure can also be defined.
4. Continuous memory allocation for structure members.
5. Structure is a user defined data type, so that it facilitates user to create variable according to the requirements.
6. Working is same as predefined data types.

Difference between Structure and Union : Refer to Q.2.

Q.34 What is dynamic memory allocation? What are the various functions in C that are used for dynamic memory allocation? Explain.

[R.T.U. 2018]

Ans. Sometimes it is desirable to create objects at run time. That is we do not inform the compiler that an object of a specified type and name needs to be created. Instead we create an object of the appropriate type ourselves at run time. This is done by allocating memory for the object that we need to create. Since the memory is allocated for objects "dynamically" (i.e., at run time), creation of such objects is known as **dynamic memory allocation**.

The C standard library provides the function `malloc()` to create object dynamically. The function takes an argument the size in bytes of the object to be created. It either returns the pointer to the created object it was able to allocate memory or returns `NULL` if not.

Function	Use of Function
<code>malloc()</code>	Allocates requested size of bytes and returns a pointer first byte of allocated space
<code>calloc()</code>	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
<code>free()</code>	deallocate the previously allocated space
<code>realloc()</code>	Change the size of previously allocated space

Syntax of `malloc()`

`ptr = (cast-type*) malloc(byte-size);`

Here, `ptr` is pointer of cast-type. The `malloc()` function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns `NULL` pointer.

Example

`ptr = (int*) malloc(100 * sizeof(int));`

This statement will allocate either 200 or 400 according to size of `int` 2 or 4 bytes respectively and the pointer points to the address of first byte of memory.

Syntax of `free()`

`free(ptr);`

This statement frees the space allocated in the memory pointed by `ptr`.

`calloc()` -

The `calloc()` stands for contiguous allocation. This function allocates memory at runtime as `malloc()` does. The difference between `malloc()` and `calloc()` is that, `malloc()` allocates single block of memory whereas `calloc()` allocates multiple blocks of memory each of same size and sets all bytes to zero.

Syntax

`ptr = (cast-type*) calloc(n, element-size);`

Here, `ptr` is pointer of cast-type. The `calloc()` function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns `NULL` pointer.

Example

`ptr = (float*) calloc(25, sizeof(float));`

This statement allocates contiguous space in memory for an array of 25 elements each of size of `float`, i.e., 4 bytes.

Program :

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) calloc(num, sizeof(int));
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```

`realloc()`

Sometimes previously allocated memory becomes insufficient or more memory is required as compared to already allocated memory. So, `realloc()` function change the previously allocated memory (allocated using `malloc()` and `calloc()`) and provides new memory blocks as required.

Syntax

`ptr = realloc(ptr, newsize);`

Here, `ptr` is reallocated with size of `newsize`.

Q.35 Write a short note on the following functions. For each function, give a program code that demonstrates its usage-

- (i) `fopen()`
- (ii) `fclose()`
- (iii) `fgetc()`
- (iv) `fseek()`

[R.T.U. 2018]

Ans. (i) **fopen() Function** : Refer to Q.1(b).

Ans. (ii) **fclose() Function** : All files opened, must be closed at the end of the program. This is done by the library function `fclose()`

The prototype of `fclose()` is given by
`int fclose(fp);`

Where, `fp` is a pointer to a FILE object of an open file*. When we close a file using `fclose()` following operations are performed :

- The characters in the buffer would be written to the file on the disk.
- At the end of file a character with ASCII value 26 would get written.
- The buffer would be removed from memory.

Though most operating systems automatically close files when the application terminates, explicitly closing all open files is a good programming practice.

Program to demonstrate `fopen()` & `fclose()` function:

```
#include <stdio.h>
#include <process.h>
void main()
{
    FILE *fp1, *fp2;
    char a;
    clrscr();
    fp1=fopen("A.txt", "r");
    if(fp1==NULL)
    {
        puts("cannot open this file");
        exit(1);
    }
    fp2=fopen("B.txt", "w");
    if(fp2==NULL)
    {
        puts("Not able to open this file");
        fclose(fp1);
        exit(1);
    }
}
```

```
a=fgetc(fp1);
a=toupper(a);
fputc(a,fp2);
}
while(a!=EOF);
fcloseall();
getch();
}
```

Ans. (iii) **fgetc() Function** : `fgetc()` function reads next `n-1` characters and stops when a new line character is encountered. The newline character is included in the array and it is terminated by `'\0'`.

The prototype of `fgetc()` is :

`char * fgetc(char * s, int n, FILE * fp);`

It reads `n-1` characters in `s`. On success it returns on end of file and on error it returns NULL.

```
#include <stdio.h>
#define MAX 15
int main()
{
    char buf[MAX];
    fgetc(buf, MAX, stdin);
    printf("string is: %s\n", buf);
    return 0;
}
```

Ans. (iv) **fseek() Function** : Refer to Q.1(a).

Q.36 Write a program to read data from the keyboard and write it to a file. Read the contents stored in the file and display it on the screen.

[R.T.U. 2018]

```
Ans: #include <stdio.h>
int main()
{
    FILE *fp; /* file pointer */
    char fName[20];
    printf("\nEnter file name to create :");
    scanf("%s", fName);

    /*creating (open) a file*/
    fp=fopen(fName, "w");
    /*check file created or not*/
    if(fp==NULL)
    {
```

```

printf("File does not created!!!");
exit(0); /*exit from program*/
}

printf("File created successfully.");
/*writting into file*/
putc('A',fp);
putc('B',fp);
putc('C',fp);
printf("\nData written successfully.");
fclose(fp);
/*again open file to read data*/
fp=fopen(fName,"r");
if(fp==NULL)
{
    printf("\nCan't open file!!!");
    exit(0);
}
printf("Contents of file is :\n");
printf("%c",getc(fp));
printf("%c",getc(fp));
printf("%c",getc(fp));
fclose(fp);
return 0;
}

```

Q.37 Write a program using pointers to insert a value in an array. [R.T.U. 2018]

Ans. #include <stdio.h>
int main()
{
 int data[5], i;
 printf("Enter elements: ");
 for(i = 0; i < 5; ++i)
 scanf("%d", data + i);
 printf("You entered: \n");
 for(i = 0; i < 5; ++i)
 printf("%d\n", *(data + i));
 return 0;
}

Output:

Enter elements: 11

24

33

51

46

You entered:

11

24

33

51

46

Q.38 Explain the difference between a null pointer and a void pointer. What do you mean by pointer arithmetic? [R.T.U. 2018]

OR

Discuss void pointer and its utility in brief.

[R.T.U. 2016]

Ans. Null Pointer : Null pointer is a pointer which is pointing to nothing. In case, if we don't have address to be assigned to a pointer, then we can simply use NULL.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Null Pointer
```

```
    int *ptr = NULL;
```

```
    printf("The value of ptr is %u", ptr);
```

```
    return 0;
```

```
}
```

Void Pointer : We know that pointers are used to point to different data types. A float pointer points to float variables, int type pointer points to integer variables, a char type pointer points to character variables. In C, there is a general purpose pointer that can point to any data types and is known as void pointer. The void pointer is a generic pointer that can represent any pointer type. The syntax of its declaration is given below :

```
void *vptr;          /*pointer to void */
```

In C, pointers to void cannot be directly dereferenced like other pointer variables by using *, the indirection operator. A suitable typecast is must prior to dereferencing a pointer to void as given below :

```

      └── dereferencing operator
      *
    *((type *)vptr)
      └── type cast

```

Here, type refers to any valid C data type.

Null pointer is a special reserved value of a pointer. A pointer of any type has such a reserved value. Formally, each specific pointer type (int *, char * etc.) has its own dedicated null-pointer value. Conceptually, when a pointer has that null value it is not pointing anywhere.

Void pointer is a specific pointer type - void * - a pointer that points to some data location in storage, which doesn't have any specific type. So, once again, null pointer is a value, while void pointer is a type. These concepts are really different and non-comparable.

Pointer Arithmetic:

Pointer arithmetic is slightly different from arithmetic we normally use in our day to day life. There are four arithmetic operators that can be used on pointers: ++, --, +, and - operators.

To understand pointer arithmetic, let us consider that ptr is an integer pointer which points to the address 1000. Assuming 32-bit integers, let us perform the following arithmetic operation on the pointer -

ptr++

After the above operation, the ptr will point to the location 1004 because each time ptr is incremented, it will point to the next integer location which is 4 bytes next to the current location. This operation will move the pointer to the next memory location without impacting the actual value at the memory location. If ptr points to a character whose address is 1000, then the above operation will point to the location 1001 because the next character will be available at 1001.

Pointer Expression	How it is evaluated ?
ptr = ptr + 1	ptr => ptr + 1 => 1000 + 1*4 => 1004
ptr++ or ++ptr	ptr++ => ptr + 1 => 1004 + 1*4 => 1008
ptr = ptr + 5	ptr => ptr + 5 => 1008 + 5*4 => 1028
ptr = ptr - 2	ptr => ptr - 2 => 1028 - 2*4 => 1020
ptr-- or --ptr	ptr => ptr + 2 => 1020 + 2*4 => 1028

For example:

```
int main()
{
    const char *p = "Hello";
    while(*p++)
        printf("%c", *p);
    return 0;
}
```

Output:
Hello

Q.39 Write a program in 'C' to read characters one by one and display it back. [R.T.U. 2016]

Ans.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char inputString[100], c;
    int index = 0;
    printf("Enter a string\n");
    while((c = getchar()) != '\n') /* Read string from
        user using getchar inside while loop */
    {
        inputString[index] = c;
        index++;
    }
    inputString[index] = '\0';

    index = 0;
    while(inputString[index] != '\0')
    {
        putchar(inputString[index]); /* Print string stored in
            inputString using putchar */
        index++;
    }
    getch();
    return 0;
}
```

Output -

Enter a string
Neelkanth
Neelkanth

Q.40 Write a 'C' program to Input and display data in following structure.

Roll - No
Student - Name
Data - of - Birth
Course
Semester

[R.T.U. 2016]

Ans.

```
#include <stdio.h>
int main()
```

PPS.56

statements are input/output statements, these are enclosed within Input/Output box.

Q.44 Write a program to check whether the input number is prime or not.

[R.T.U. 2014, 2011, Raj. Univ. 2004]

```
Ans. #include<stdio.h>
#include<conio.h>
void main()
{
    int f=0,n,i;
    clrscr();
    printf("\nEnter a number");
    scanf("%d",&n);
    for(i=2;i<n;i++)
    {
        if(n%i==0)
        {
            f=1;
            break;
        }
    }
    if(f==0)
    printf("\nNumber is Prime");
    else
    printf("\nNumber is Not Prime");
    getch();
}
```

Output :

Enter a number 5
Number is Prime

Q.45 Write a program to print reverse of a number using function.

[R.T.U. 2014]

Ans. /*c program for reverse number using user define function- rev */

```
#include<stdio.h>
int rev(int);
int main()
{
    int num,res; // res = result
    printf("Enter any number: ");
    scanf("%d", &num);
    res = rev(num);
    printf("Reverse order number = %d", res);
}
```

Programming for Problem Solving

```
return 0;
}
int rev(int n)
{
    int r=0;
    for(; n>1; n=n/10)
        r = r * 10 + n%10;
    return r;
}
```

The output of above program would be :
Enter any number : 7941
Reverse order number = 1497

Q.46 Is there any difference between header file and library file? Explain.

[R.T.U. 2013]

Ans. Header Files : These are the files that are included at the top of any program. If we use any function inside a program, then the header file containing declaration or definition of that function, has to be included. Like printf() is defined in stdio.h. So, we must include it (by #include in order to use printf()).

Library Files : These are the files which the compiler uses in order to define the functions which have been used in the program and had been declared inside the header file. Like, printf() has its complete definition, i.e., how it will work etc. in an I/O library. So, the compiler uses that library to get the machine code for printf().

Differences

- Header files are TEXT files while library files are BINARY. This means, we can read and modify the header file but not the library file.
- Header file is in C language while the library file is in machine language.
- Header file has to be included by the programmer while the compiler automatically relates the library file(s) with the program.

Q.47 (a) List arithmetic and logical operators available in C.

(b) Write a program to find sum of following terms of a series. $1^2, 2^2, 3^2, 4^2, \dots$

[R.T.U. 2012]

Ans. (a) Arithmetic Operators

Arithmetic operators are used to perform numeric calculations. These can operate on any built in data type allowed in C. These operators are listed below in the table.1:

Table 1 : Arithmetic Operators

Operator Symbol	Meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulo division (applicable only on int data type)

Logical Operators

Logical operators are used to combine two expressions. These operators check logical relation between the two or more expressions. Depending upon the true and false nature of the expression they assigned value 1 and 0. Logical operators are listed below in the table 2.

Table 2 : Logical Operators

Logical Operators	Meaning
&&	AND
	OR
!	NOT

The logical operators && and || are used when we want to test more than one condition and make decisions.

Example : $z > y$ && $x = 10$

An expression of this kind, which combines two or more relational expression is termed as a logical expression or a compound relational expression.

1. Logical AND (&&) Operator

Logical AND works with two or more operands and gives the result true if all the operands in the condition have true value, otherwise it provides the result false.

Table 3 : Logical AND (&&)

Operand 1	Operand 2	Result
False	False	False
False	True	False
True	False	False
True	True	True

Example : Assume we want to compare age and salary of an employee

$\text{age} > 25$ && $\text{salary} > 10000$

Here, from the above example if age of the employee is greater than 25 and salary is greater than 10000 then the result will be True (1) only, otherwise result will be False (0).

2. Logical OR (||) Operator

Logical OR works with two or more operands and gives the result true if one of the operands in the condition have true value, otherwise it provides the result false.

Table 4 : Logical OR (||)

Operand 1	Operand 2	Result
False	False	False
False	True	True
True	False	True
True	True	True

Example : Assume that we want to compare age and salary of an employee

$\text{age} > 25$ || $\text{salary} > 10000$

Here, in the above example the condition will be true in all the following cases :

(i) If age of employee is greater than 25, salary can be anything.

(ii) If salary of employee is greater than 10000, now age can be anything.

(iii) If both age is greater than 25 and salary greater than 10000.

(iv) Only, if age is not greater than 25 and salary is not greater than 10000 then the condition will result false.

3. Logical Not (!) Operator

This is a unary operator. This operator convert a true value into false and a false value into true.

Table 5 : Logical Not (!)

Condition	Result
True	False
False	True

Example : $!(a > b)$

Here, if a is greater than b then the actual result is true, but we have used ! operator before the expression so the result will be false. And if a is not greater than b then the result will be false, but ! will convert it into true.

Ans. (b) # include <stdio.h>

include <conio.h>

void main ()

{

int i, n;

long sum = 0;

clrscr ();

printf ("Enter the value of n =");

scanf ("%d", &n);

for (i = 1; i <= n; i++)

sum = sum + i * i;

printf (" sum of series = %ld", sum);

getch ();

}

Output

Enter the value of n = 5

Sum of series = 55.

Q.48 What are the advantages and disadvantages of C language? [R.T.U. 2011]

Ans. Advantages of C Language

- C is a building block for many other currently known languages. Take a look at Python for example a fully Object-Oriented High-Level programming language. It is written in C (perhaps C++ too). That tells you if you ever want to know what is going on under the hood in other languages; understanding C and how it works is essential.
- C is a compiled language versus an interpreted language. Explained simply, this means that the code is compacted into executable instruction (in the case of windows anyway) rather than being "translated" on the fly at run time. This feature also lends heavily to the speed of C programs.
- A lot of libraries are written in C.
- The main advantages of C language is that there is not much vocabulary to learn, and that the programmer can arrange for the program is very fast. C programming language is very easier to learn. C programming language is still a practical and compact language. It comprises a good semantic. Syntax of C is clear also. C language is very near to assembly programming i.e. the hard use of pointers for example is a very powerful mechanism.
- C has features that allow the programmer to organize programs in a clear, easy, logical way. For example, C allows meaningful names for variables without any loss of efficiency, yet it gives a complete freedom of programming style, including flexible ways of making decisions, and a set of flexible commands for performing tasks repetitively (for, while, do).
- C is a portable language.

Disadvantages of C Language

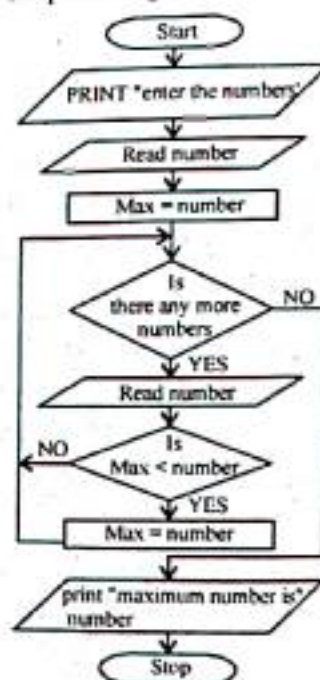
- C does not have OOPS feature that's why C++ is developed.
- There is no runtime checking in C language.
- There is no strict type checking (for e.g. we can pass an integer value for the floating data type).
- C does not have the concept of namespace.
- C does not have the concept of constructors and destructors.

Q.49 Draw the flow chart and write a C program to find out largest number in a given list of 10 numbers. [R.T.U. 2010, Raj. Univ. 2002]

Ans. #include<stdio.h>

void main()

```
{
    int list[ ] = {10,40,75,35,46,67,23,96,34,35};
    int max,temp;
    max = list[0];
    for(int i = 0;i<10;i++)
    {
        temp = list[i];
        if (max<temp)
            max = temp;
    }
    printf("Largest Number is: %d",max);
    //Output: Largest Number is: 96
}
```



PART C

Q.50 What is recursion? Write a program to find factorial of a given number using recursion. [R.T.U. 2019]

Ans. Recursion : Recursion is the process of repeating code defined by a function itself. In recursion, the function

function calls itself so that instructions can be executed more than one time.

Example :

```
main()
{
    Body of main() function
    ...
    ...
    main(); /*Recursive call to main() function */
}
```

Recursion is very useful to solve various mathematical problems like factorial calculation, printing of Fibonacci series, Solving Tower of Hanoi Problem, GCG calculation etc. While implementing recursive functions in the program, the loop control statements are not used like for, while or do-while.

Calculate factorial using recursion.

```
#include <stdio.h>
int factorial(int n)
{
    if(n <= 1)
    {
        return 1;
    }
    return n * factorial(n-1);
}
int main()
{
    int n = 5;
    printf("Factorial of %d is %d\n", n, factorial(n));
    return 0;
}
```

Q.51 Write a C program to find the sum of the series-

$$S = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^n}{n!}$$

[R.T.U. 2019]

Ans. #include <stdio.h>

void main()

```
{
    float x, sum, no_row;
    int i, n;
    printf("Input the value of x :");
    scanf("%f", &x);
    printf("Input number of terms :");
```

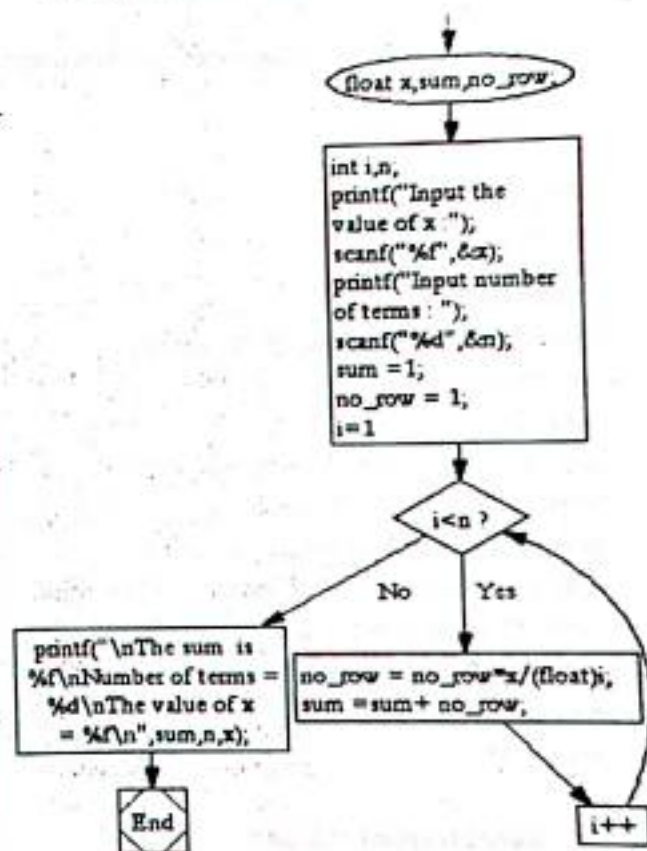
```
scanf("%d", &n);
sum = 1; no_row = 1;
for(i=1; i<n; i++)
{
    no_row = no_row * x / (float)i;
    sum = sum + no_row;
}
printf("\nThe sum is : %f\n", sum);
}
```

Output:

Input the value of x :3

Input number of terms : 5

The sum is : 16.375000



Q.52 Programming on structures/pointers-

(a) Define a structure data type called time_struct containing 3 members called hour, minute and second. Develop a program that would assign values to the individual members and display the time in the form 16:40:30 [R.T.U. 2019]

(b) Write a program in C to swap Three number using pointer. [R.T.U. 2019]

PPS.60

Ans.(a) #include <stdio.h>

```
struct time_struct
{
    int hour;
    int minute;
    int second;
};
void main()
{
    struct time_struct time;
    clrscr();
    time.hour=16;
    time.minute=40;
    time.second=30;
    printf("%d:%d:%d",time.hour,time.minute,time.second);
    getch();
}
```

Output

16:40:30

Ans.(b) #include <stdio.h>

```
void cyclicSwap(int *a, int *b, int *c);
int main() {
    int a, b, c;
    printf("Enter a, b and c respectively: ");
    scanf("%d%d%d", &a, &b, &c);
    printf("Value before swapping:\n");
    printf("a = %d \nb = %d \nc = %d\n", a, b, c);
    cyclicSwap(&a, &b, &c);
    printf("Value after swapping: \n");
    printf("a = %d \nb = %d \nc = %d", a, b, c);
    return 0;
}
void cyclicSwap(int *n1, int *n2, int *n3) {
    int temp;
    // swapping in cyclic order
    temp = *n2;
    *n2 = *n1;
    *n1 = *n3;
    *n3 = temp;
}
```

Output

Enter a, b and c respectively :

```
1
2
3
Value before swapping:
a = 1
b = 2
c = 3
Value after swapping:
a = 3
b = 1
c = 2
```

Q.53 Explain the concept of File handling in C language. Write a Program in C language to copy the data from source file to destination file and display the same on console. [R.T.U. 2019]

Ans. C File Handling : A file is a container in computer storage devices used for storing data.

Need of Files :

- When a program is terminated, the entire data is lost. Storing in a file will preserve data even if the program terminates.
- If we enter a large number of data, it will take a lot of time to enter them all. However, if a file containing all the data, user can easily access the contents of the file using a few commands in C.
- User can easily move data from one computer to another without any changes.

Types of Files

When dealing with files, there are two types of files:

1. Text files
2. Binary files

1. Text files : Text files are the normal .txt files. User can easily create text files using any simple text editors such as Notepad.

When user open those files, we'll see all the contents within the file as plain text. User can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

2. Binary files : Binary files are mostly the .bin files in computer. Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold a higher amount of data, are not readable easily, and provides better security than text files.

File Operations

1. Creating a new file
2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

Working with files

When working with files, user need to declare a pointer of type file. This declaration is needed for communication between the file and the program.

`FILE *fptr;`

Opening a file - for creation and edit

Opening a file is performed using the `fopen()` function defined in the `stdio.h` header file.

The syntax for opening a file in standard I/O is:

`fptr = fopen("filename", "mode");`

For example,

`fopen("E:\\cprogram\\newprogram.txt", "w");`

`fopen("E:\\cprogram\\oldprogram.bin", "rb");`

- Let's suppose the file `newprogram.txt` doesn't exist in the location `E:\\cprogram`. The first function creates a new file named `newprogram.txt` and opens it for writing as per the mode 'w'.

The writing mode allows you to create and edit (overwrite) the contents of the file.

- Now let's suppose the second binary file `oldprogram.bin` exists in the location `E:\\cprogram`. The second function opens the existing file for reading in binary mode 'rb'.

The reading mode only allows to read the file, but cannot write into the file.

a	Open for append. Data is added to the end of the file.	If the file does not exist, it will be created.
ab	Open for append in binary mode. Data is added to the end of the file.	If the file does not exist, it will be created.
r+	Open for both reading and writing.	If the file does not exist, <code>fopen()</code> returns NULL.
rb+	Open for both reading and writing in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.
w+	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb+	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a+	Open for both reading and appending.	If the file does not exist, it will be created.
ab+	Open for both reading and appending in binary mode.	If the file does not exist, it will be created.

Closing a File

The file (both text and binary) should be closed after reading/writing.

Closing a file is performed using the `fclose()` function.

`fclose(fptr);`

Here, `fptr` is a file pointer associated with the file to be closed.

Reading and writing to a text file

For reading and writing to a text file, we use the functions `fprintf()` and `fscanf()`.

They are just the file versions of `printf()` and `scanf()`. The only difference is that `fprint()` and `fscanf()` expects a pointer to the structure `FILE`.

`#include <stdio.h>`

`#include <stdlib.h> // For exit()`

`int main()`

{

`FILE *fptr1, *fptr2;`

`char filename[100], c;`

`printf("Enter the filename to open for reading\n");`

`scanf("%s", filename);`

`// Open one file for reading`

Mode	Meaning of Mode	During Inexistence of file
r	Open for reading.	If the file does not exist, <code>fopen()</code> returns NULL.
rb	Open for reading in binary mode.	If the file does not exist, <code>fopen()</code> returns NULL.
w	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.

```

fptr1 = fopen(filename, "r");
if(fptr1 == NULL)
{
    printf("Cannot open file %s \n", filename);
    exit(0);
}
printf("Enter the filename to open for writing \n");
scanf("%s", filename);
// Open another file for writing
fptr2 = fopen(filename, "w");
if(fptr2 == NULL)
{
    printf("Cannot open file %s \n", filename);
    exit(0);
}
// Read contents from file
c = fgetc(fptr1);
while(c != EOF)
{
    fputc(c, fptr2);
    c = fgetc(fptr1);
}
printf("\nContents copied to %s", filename);
fclose(fptr1);
fclose(fptr2);
return 0;
}

```

Output:

Enter the filename to open for reading
a.txt
Enter the filename to open for writing
b.txt
Contents copied to b.txt

Q.54 What is array? Explain types of array with suitable example. [R.T.U. 2019]

Ans. Array : An array is basically a group or collection of more than one elements. We have only stored a single value in a variable and if we store another value in that

variable then the previous is lost. So if we want to store more than one value under a single name at the same time during program execution then C provides us the facility to use arrays.

There are some characteristics that an array has, are :

- An array is a collection of more than one element.
- All the elements that an array store will be of same data type.
- The number of elements or variables that an array can hold is called the size of the array.
- All the elements of an array are stored in continuous memory locations.
- Once the size of array has declared then we cannot modify that at run time.

Classification of Arrays : There are three types of Array :

- One Dimensional Array
- Two Dimensional Array
- Three Dimensional Array

1. One Dimensional Array : This type of array supports one row and multiple columns to store more than one element. Declaration, initialization, and memory allocation of one dimensional array are discussed here :

(i) Array Declaration : Like other variables an array need to be declared so that the compiler will know what kind of an array and how large an array we want.

Syntax :

data type array name [size of array];

Example : int num[100];

Here, in the above example there are three parts of array declaration, int specifies the type of the variable, num specifies the name of the variable as well as name of the memory location. The [100] specifies the size of the array and here it is telling that we can store 100 elements of int type in the array num.

We can also declare array of other data types like :

float salary [50];

char name [10];

(ii) Array Elements in Memory : As we have already seen that elements of array are stored continuously in memory one after one. Consider the following array declaration :

```
int arr[8];
```

arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]	arr[6]	arr[7]	→ Array Index
12	24	-40	11	2	0	90	77	→ Values of array
2000	2002	2004	2006	2008	2010	2012	2014	→ Addresses of elements

Programming for Problem Solving

Here, in the above array declaration total 16 bytes of memory is allocated, because int data type occupies 2 bytes in memory. All the elements of array contain garbage value because we have not initialized the array elements. Index of array is used to retrieve and store any value at a particular position of the array. Array index always starts from 0 and ends with (size - 1).

(iii) **Array Initialization** : At the time of declaration of an array all the elements of array contain garbage value, until we initialize any particular value to the elements of array. The process of assigning values at the time of declaration of an array is called Array Initialization.

Example :

```
int num [6] = { 2, 4, 12, 5, 45, 5};
int n [ ] = { 2, 4, 12, 5, 45, 5};
float value [ ] = { 12.3, 34.2, -23.4, -11.3};
```

Till we have not assigned any value to array, it contains garbage. If the array is initialized where it is declared mentioning one dimension, then the size of the array becomes optional part as in the above examples.

(iv) **Accessing Elements of an Array** : To access the elements of an array, index is used. Index of array begins with 0 and ends with (size - 1). So if we want to access a particular element of an array then the statement would be like this :

Example :

```
num[1]; To access second element
num[5]; To access sixth element
num[0]; To access first element
```

(v) **A Simple Program Using an Array**

```
main ()
{
    int i;
    int marks [10];
    /* array declaration */
    float avg, sum = 0;
    printf("Enter marks of 10 students");
    for ( i = 0; i < 10; i++)
    {
        printf( " Enter Marks");
        scanf( "%d", &marks [i]);
        /* read marks from user */
    }
    for ( i = 0; i < 10; i++)
        sum = sum + marks [i];
    /* add marks of 10 students */
}
```

```
avg = sum / 10;
printf( "\n Average Marks = %f", avg);
}
```

2. Two Dimensional Array

A two dimensional (2-D) array is a collection of more than one rows and columns. Two dimensional array can be called an array of several 1-D arrays. Generally 2-D arrays are used to solve the problems related to matrix manipulation.

(i) **Declaration**

Syntax :

```
data type array name [row_size] [col_size];
```

Example :

```
int mat [3] [3];
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

A 2-D array of 3×3

(ii) **Initializing 2-D Array** : A 2-D array can be initialized in the following different ways :

(i) `int s[4][2] = {`

`{1234, 56},`

`{1212, 33},`

`{1342, 32},`

`{2145, 46}`

`};`

(ii) `int s[4][2] = { 1234, 56, 1212, 33, 342, 32, 2145, 46};`

(iii) `int arr[2][3] = { 1, 2, 3, 4, 5, 6};`

(iv) `int arr[ ] [3] = { 1, 2, 3, 4, 5, 6};`

It is important to remember that, while initializing a 2-D array it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional, as in example (iv) row size has not given but the compiler will automatically assume it by 2.

(iii) **Memory Map of a 2-Dimensional Array** : In memory a 2-D array does not contain rows and columns. In memory both 1-D and 2-D arrays are stored continuously. So the arrange of memory allocation given below of a 2-D array is conceptually true, and the elements of array are stored row wise in memory.

Example :

int s[4][2] = {1234, 56, 1212, 33, 342, 32, 2145, 46};

s[0][0]	s[0][1]	s[1][0]	s[1][1]	s[2][0]	s[2][1]	s[3][0]	s[3][1]
1234	56	1212	33	342	32	2145	46
2000	2002	2004	2006	2008	2010	2012	2014

→ Array Index
→ Values of array
→ Addresses of elements

(iv) Accessing Elements of 2-D Array : In two dimensional array, if we want to access any element then two indexes, one for row and second for column are required.

Example :

s[1][1] : It will access element of 2nd row and 2nd column.

s[3][2] : It will access element of 4th row and 3rd column.

Program : Write a program to read a 3*3 matrix and print its values.

Sol.: #include<stdio.h>

#include<conio.h>

void main()

{

int a[3][3], i, j;

clrscr();

printf("\nEnter elements of matrix");

for(i=0; i<3; i++)

{ for(j=0; j<3; j++)

scanf("%d", &a[i][j]);

}

printf("\nYou have entered the following values\n");

for(i=0; i<3; i++)

{

for(j=0; j<3; j++)

printf("%d\t", a[i][j]);

printf("\n");

}

}

Output:

Enter elements of matrix

1

2

3

4

5

6

7

8

9

You have entered the following values

1

2

3

4

5

6

7

8

9

Q.55 What do you understand by file handling? Explain various file handling functions used in C language. [R.T.U. 2019]

Ans. Handling Files in C : C provides a rich library of functions for manipulating files. There are a wide range of functions for working with text files and binary files in either the sequential access or random access mode.

All the functions work with a structure FILE declared in "stdio.h".

All functions that work with files expect a pointer to a FILE object. Note that this pointer is different from the file pointer.

The following statement declares a pointer to a FILE object :

FILE *fp;

This pointer can be used with functions that work with files.

(i) fopen() Function : The first operation needed to be done before processing data in a file is to open the file. This is done by the fopen() function.

The prototype of fopen() is given below :

FILE * fopen(char * filename, char * mode);

Where filename specifies the name of the file on the secondary storage and mode specifies whether the file should be opened for reading or writing or for both.

The function fopen() returns a pointer to a FILE object if it could successfully open the file. It returns NULL if it could not. It is always good to test the value returned by fopen() function. The following statements attempt to open a file called "Student. dat".

Example :

#include <stdlib.h>

#include <stdio.h>

FILE *fp;

```
fp = fopen("Student. dat", "r")
if (fp == NULL)
{
    printf("\n Error opening file Student. dat");
    exit(1);
}
```

The statement,

```
fp = fopen("Student. dat", "r")
```

opens the file "Student. dat" in the read mode. If `fopen()` was able to successfully open the file, it returns a pointer to the FILE object which is saved in `fp`. Otherwise it returns NULL. The if statement tests the value of `fp` and takes appropriate action.

It is possible to combine the file opening and test operations into a single statement. This is done as follows :

```
if ((fp = fopen("Student. dat", "r")) == NULL)
{
    printf("\n Error opening file "Student. dat");
    exit(1);
}
```

The second argument to `fopen()` mode must be one of the following values. The valid modes and their meaning are given below :

Mode	Description
"r"	Opens an existing file for reading only.
"w"	Opens a new file for writing only. If a file with the specified name already exists, its previous contents are erased.
"a"	Opens an existing file for appending (i.e., for adding new information at the end of the file). A new file will be created if the file with the specified filename does not exist. Any previous contents are preserved.
"r+"	Opens an existing file for both reading and writing. This mode is also known as update mode.
"w+"	Opens a new file for both reading and writing. If the file with the specified name already exists, its previous contents are erased.
"a+"	Opens an existing file for both reading and writing. A new file will be created if the file with the specified name does not exist. Any previous contents are preserved. All write operations will be done only at the end of the file.

By default all the above modes assume that the specified file is a text file. In order to specify that the file is to be opened in the binary mode, a "b" needs to be appended to the mode.

For example, to open a binary file in the read only mode, the mode should be specified as "rb". To specify that the file should be opened in the update mode, then the mode should be specified as "r+b".

It is possible to open more than one file. In such a case a pointer to the FILE object needs to be maintained for each of these files. The maximum number of files that can be opened by a program is usually operating system dependent. In the case of DOS, the maximum number is specified by the FILES setting in CONFIG.SYS.

The following program segment opens two files one for reading and one for writing :

Example :

```
# include <stdlib.h>
# include <stdio.h>
FILE *ReadFile, *WriteFile;
if ((ReadFile = fopen("Infile. txt", "r")) == NULL)
{
    printf("\n Error opening Infile. txt");
    exit(1);
}
```

Returns EOF(-1) if any error occurs, else returns zero.

```
if ((writeFile = fopen("Outfile.txt", "w")) == NULL)
```

```
{
    printf("\n Error opening Outfile.txt");
    fclose(Read File);
    exit(2);
}
```

(ii) `fclose()` Function : All files opened, must be closed at the end of the program. This is done by the library function `fclose()`

The prototype of `fclose()` is given by
`int fclose(fp);`

Where, `fp` is a pointer to a FILE object of an open file. When we close a file using `fclose()` following operations are performed :

- The characters in the buffer would be written to the file on the disk.
- At the end of file a character with ASCII value 26 would get written.

- The buffer would be removed from memory.

Though most operating systems automatically close files when the application terminates, explicitly closing all open files is a good programming practice.

(iii) fprintf() Function : fprintf() function can be used to write formatted text into a file. The fprintf() function works only with text file.

The prototype of fprintf() is very similar to that of printf() except for one difference. The fprintf() function needs one additional parameter- the file to which the output should be sent. The prototype of fprintf() is :

`int fprintf(FILE *fp, const char *fmt-str, ...);`
where, fp is a pointer to the FILE object representing the file to which the output should be written. fmt-str is the format string and the ellipses represent the arguments whose values need to be embedded into the fmt-str.

Note that fmt-str is a pointer to a constant character string.

The fprintf() function returns the number of characters written in the file, or a negative value if an error occurred.

(iv) fscanf() Function : fscanf() function reads information from the specified file. It works very similar to the scanf() function. fscanf() function works only with text files. The prototype of fscanf() is very similar to the scanf() function with one difference. It needs an additional parameter- a pointer to a FILE object representing the file from which input needs to be read.

The prototype of fscanf() is-

`int fscanf(FILE *fp, const char *fmt-str, ...);`

fscanf() function returns the number of input items that were read and assigned to respective variables. On error it returns EOF.

(v) fgetc() Function : It is used to read a character from the specified file.

The prototype of fgetc() function is :

`int fgetc(FILE *fp);`

It returns the next character from the file. On end of file or error it returns EOF.

(vi) fputc() Function : The fputc() function writes a character to the file. It returns back the character written on success and on error it returns EOF.

The prototype of fputc() is-

`int fputc(char c, FILE *fp);`

(vii) fgets() Function : fgets() function reads next n - 1 characters and stops when a new line character is encountered. The newline character is included in the array and it is terminated by '\0'.

The prototype of fgets() is :

`char * fgets(char *s, int n, FILE *fp);`

It reads n - 1 characters in s. On success it returns on end of file and on error it returns NULL.

(viii) fputs() Function : fputs() function can be used to write a string in the file specified by fp.

The prototype of fputs() is:

`int fputs(const char *s, FILE *fp);`

On success fputs() returns a non-negative value and on error it returns EOF.

(ix) getw() Function : getw() function can be used to read an integer from a file. The prototype of getw() is :

`int getw(FILE *fp);`

On success, it returns the integer. On error or end of file, it returns EOF. The getw() function works with text files only.

(x) putw() Function : putw() function can be used to write an integer into a file. The prototype of putw() is :

`int putw(int i, FILE *fp);`

On success, it returns the integer i and on error it returns EOF.

(xi) fcloseall() Function : If multiple files are opened in a program then all of them can be closed in one statement only by using fcloseall() function. The prototype of fcloseall() is :

`int fcloseall(void);`

On success it returns the number of files closed by it and on error it returns EOF.

(xii) feof() macro : While reading from a file usually we start from the beginning of the file and read data till end of file is encountered (sequential access).

We can use feof() macro, declared in "stdio.h" to test whether end of file is encountered or not. The prototype of feof() is :

`int feof(FILE *fp);`

It returns non zero value if EOF is encountered else it returns 0.

Usually we use feof() inside a while loop as :

```
while(! feof(fp))
{
    /*do something */
}
```

Q.56 What is looping? Explain types of loop available in C. Also write a program to check the input number is prime or not. (R.T.U. 2014)

Looping : For solving variable size problems, programs need to execute the same set of instructions many times to determine the results. This process of repeating the same sequence of steps is referred to as **iteration or Looping**.

In every loop control we have three major tasks :

(a) **Initialization:** In every loop we use a loop counter which is used to execute the loop number of times according to the condition. Before using this counter we have to initialize it with a constant value.

(b) **Termination Condition:** The number of times the code will execute must be finite. Otherwise it becomes an infinite loop. The loop continues until some required logical condition has been satisfied. So the loop counter which we have initialized executes the loop statements until the required condition has not fulfilled.

(c) **Increment / Decrement:** To move loop counter ahead we have to increment or decrement loop counter according to initialization and termination condition.

Loop Statement

There are 3 methods by using of loop which we can repeat a part of a program in C :

- Using a **for** statement
- Using a **while** statement
- Using a **do-while** statement

1. for Loop

for is the most popular looping control. This loop allows us to specify all three things we have read above in a single line :

- Initializing loop counter
- Testing condition
- Increment or decrement to jump for next iteration

Syntax :

for (initialization ; termination condition ; increment/decrement)

```
{
    statement 1;
    statement 2;
    _____
    _____
    statement n;
}
```

Example :

```
main ()
{
    int i;
    for ( i = 1 ; i <= 10 ; i = i + 1 )
        printf( "%d\n", i );
}
```

Here, the initialization, testing and increment of loop counter is done in the header part of **for** statement. Instead of $i = i + 1$, the statements $i++$ or $i += 1$ can also be used.

2 while Loop

while loop is an entry control or top tested loop. It means the test condition is evaluated first and if the condition is true first time then the body of the loop is executed. After execution of body of loop control again goes to the test condition, if this time the test condition is true then body of loop is executed again and the process goes on until the test condition becomes false.

Syntax :

```
initialization;
while( test condition )
{
    statement 1;
    statement 2;
    _____
    _____
    statement n;
    increment / decrement;
}
```

Example :

```
main ()
{
    int i = 1;
    while ( i <= 10 )
    {
        printf( "%d\n", i );
        i++;
    }
}
```

The **while** must test a condition that will eventually become false, otherwise the loop would be executed forever, infinitely.

3. do-while Loop

do-while loop is an exit control or bottom tested loop. This loop almost works like while loop but there is a difference, that it first executes the body of the loop and then checks whether the condition is true or not. If the condition is false at the first time, then it will also execute the statements at least once.

Syntax :

```

initialization;
do
{
    statement 1;
    statement 2;
    _____
    statement n;
    increment / decrement;
} while( Test condition);

```

Example :

```

main()
{
    int a = 5 ;
    while ( a<5 )
    {
        printf("Hello World
\n");
    }
}

```

Here, the condition fails for the first time itself the `printf()` will not get executed.

```

main()
{
    int a = 5 ;
    do
    {
        printf("Hello World
\n");
    }while(a<5);
}

```

In this example, the `printf()` would be executed once, since first time body of the loop is executed and then the condition is tested.

Input Number is Prime or Not : Refer to Q.44.

Q.57 Discuss the difference between call by value and call by reference parameter passing technique with suitable examples. [R.T.U. 2018]

Ans. Compare Chart

Basic for comparison	Call by value	Call by Reference
Basic	A copy of the variables is passed.	A variables itself is passed.
Effect	Change in a copy of variable doesn't modify the original value of variable outside the function.	Change in the variable affect the value of variable outside the function also.
Calling parameters	Function_name (variable_name 1, variable_name2, ...);	Function_name (& variable_name1 and variable_name2 ...); //in case of object object.func_name (object);
Receiving parameters	Type function_name (type variable_name 1, type variable_name 2, ...); {...}	Type function_name (type * variable_name1, type * variable_name2,...); {...} // in case of object type function_name (class type object_name) {...}
Default calling	Primitive type are passed using "call by value".	Object are implicitly passed using "call by reference".

Call by Value : Refer to Q.24.

Call by Reference : Refer to Q.24.

Q.58 Explain the concept of file handling in C language. Write a program to copy the data from source file to destination file. [R.T.U. 2018]

Ans. File is a collection of bytes that is stored on secondary storage devices like disk. There are two kinds of files in a system. They are,

- Text files (ASCII)** : Text files contain ASCII codes of digits, alphabetic and symbols.
- Binary files** : Binary file contains collection of bytes (0's and 1's). Binary files are compiled version of text files.

Basic File Operations in C Programming :

There are 4 basic operations that can be performed on any files in C programming language. They are,

- (i) Opening/Creating a file
- (ii) Closing a file
- (iii) Reading a file
- (iv) Writing in a file

Function	Description
<code>fopen()</code>	create a new file or open an existing file
<code>fclose()</code>	closes a file
<code>getc()</code>	reads a character from a file
<code>putc()</code>	writes a character to a file
<code>fscanf()</code>	reads a set of data from a file
<code>fprintf()</code>	writes a set of data to a file
<code>getw()</code>	reads an integer from a file
<code>putw()</code>	writes an integer to a file
<code>fseek()</code>	set the position to desired point
<code>tell()</code>	gives current position in the file
<code>rewind()</code>	set the position to the beginning point

Opening or Creating a file

The `fopen()` function is used to create a new file or to open an existing file.

General Syntax :

`*fp = FILE *fopen(const char *filename, const char *mode);`

Here "`*fp`" is the FILE pointer (`FILE *fp`), which will hold the reference to the opened (or created) file.

"filename" is the name of the file to be opened and mode specifies the purpose of opening the file. Mode can be of following types :

Mode	Description
<code>r</code>	opens a text file in reading mode
<code>w</code>	opens or create a text file in writing mode.
<code>a</code>	opens a text file in append mode
<code>r+</code>	opens a text file in both reading and writing mode
<code>w+</code>	opens a text file in both reading and writing mode
<code>a+</code>	opens a text file in both reading and writing mode
<code>rb</code>	opens a binary file in reading mode
<code>wb</code>	opens or create a binary file in writing mode

<code>ab</code>	opens a binary file in append mode
<code>rb+</code>	opens a binary file in both reading and writing mode
<code>wb+</code>	opens a binary file in both reading and writing mode
<code>ab+</code>	opens a binary file in both reading and writing mode

Program to copy the data from source file to destination file :

```
#include <stdio.h>
#include <stdlib.h> // For exit()
int main()
{
    FILE *fptr1, *fptr2;
    char filename[100], c;
    printf("Enter the filename to open for reading \n");
    scanf("%s", filename);
    // Open one file for reading
    fptr1 = fopen(filename, "r");
    if (fptr1 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }
    printf("Enter the filename to open for writing \n");
    scanf("%s", filename);
    // Open another file for writing
    fptr2 = fopen(filename, "w");
    if (fptr2 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }
    // Read contents from file
    c = fgetc(fptr1);
    while (c != EOF)
    {
        fputc(c, fptr2);
        c = fgetc(fptr1);
    }
    printf("\n Contents copied to %s", filename);
    fclose(fptr1);
```

PPS.70

```
fclose(fp2);
return 0;
}
```

Output:

Enter the filename to open for reading
a.txt
Enter the filename to open for writing
b.txt
Contents copied to b.txt

Q.59 Discuss the uses of structure and pointer to structure with suitable examples. [R.T.U. 2018]

Ans. Uses of Structure : Refer to Q.2.

Uses of Pointer to Structure

1. To pass arguments by reference. Passing by reference serves two purposes:

- (i) To modify variable of function in other.

```
#include <stdio.h>
void swap(int* x, int* y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main()
{
    int x = 10, y = 20;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

- (ii) For efficiency purpose. Example passing large structure without reference would create a copy of the structure (hence wastage of space).

2. For accessing array elements. Compiler internally uses pointers to access array elements :

```
#include <stdio.h>
int main()
{
    int arr[] = { 100, 200, 300, 400 };
    // Compiler converts below to *(arr + 2).
    printf("%d", arr[2]);
    // So below also works.
```

```
printf("%d\n", *(arr + 2));
return 0;
}
```

3. To return multiple values :

```
#include <math.h>
#include <stdio.h>
void fun(int n, int* square, double* sq_root)
{
    *square = n * n;
    *sq_root = sqrt(n);
}
int main()
{
    int n = 100;
    int sq;
    double sq_root;
    fun(n, &sq, &sq_root);
    printf("%d %f\n", sq, sq_root);
    return 0;
}
```

4. Dynamic memory allocation : We can use pointer to dynamically allocate memory. The advantage of dynamically allocated memory is, it is not deleted until we explicitly delete it.

```
#include <stdio.h>
int* createArr(int n)
{
    return new int[n];
}
int main()
{
    int* pt = createArr(10);
    return 0;
}
```

5. To implement data structures : Example : linked list, tree, etc. We cannot use C++ references to implement these data structures because references are fixed in a location (For example, we cannot traverse a linked list using references).

6. To do system level programming where memory addresses are useful : For example shared memory used by multiple threads. For more examples, see IPC through shared memory, Socket Programming in C/C++ etc.

Rank	C
1	(
	[
	-
	.
2	-
	+
	~
	!
	&
	*
	(t
	si
3	*
	/
	%
4	+
	-
5	<<
	>>

- Q.60 (a) Discuss Enumerated data type with the help of suitable code of 'C' language.
 (b) Arrange following operators in order of precedence and associativity rules of 'C'.
 +, %, <, /, *, =, !, ++, () [R.T.U. 2016]

Ans.(a) Enumerated data type : Refer to Q.19(b).

Ans.(b) Operator Precedence and Associativity

In C each operator has a precedence level associated with it. Precedence means, the sequence or order of different operators in an expression and it determines which operator will be executed first or which will second and so on. There are distinct levels of precedence and an operator may belong to one of these levels. Operator with a higher precedence is evaluated before operator with a lower precedence.

Some of the operators have same precedence level, i.e. the operators of the same precedence are evaluated either from 'left to right' or from 'right to left', depending on the level. This is known as the associativity of an operator. Like arithmetic operators have left to right associativity and assignment operators have right to left associativity.

The table provides the list of operators with their precedence level and associativity :

Table

Rank	Operator	Description	Associativity
1	()	Function expression	Left to Right
	[]	Array expression	Left to Right
	→	Structure operator	Left to Right
	.	Structure operator	Left to Right
2	-	Unary minus	Right to Left
	++, --	Increment/Decrement	Right to Left
	~	One's Complement	Right to Left
	!	Negation	Right to Left
	&	Address of	Right to Left
	*	Value at address	Right to Left
	(type)	Type cast	Right to Left
	sizeof	Size in bytes	Right to Left
3	*	Multiplication	Left to Right
	/	Division	Left to Right
	%	Modulus	Left to Right
4	+	Addition	Left to Right
	-	Subtraction	Left to Right
5	<<	Left shift	Left to Right
	>>	Right shift	Left to Right

6	<	Less than	Left to Right
	<=	Less than or equal to	Left to Right
	>	Greater than	Left to Right
	>=	Greater than or equal to	Left to Right
7	=	Equal to	Left to Right
	!=	Not equal to	Left to Right
8	&	Bitwise AND	Left to Right
9	^	Bitwise exclusive OR	Left to Right
10		Bitwise inclusive OR	Left to Right
11	&&	Logical AND	Left to Right
12	:	Logical OR	Left to Right
13	?:	Conditional/ Ternary	Right to Left
14	=	Assignment	Right to Left
	*= /= %=		Right to Left
	+= -=		Right to Left
	&=		Right to Left
	^= =		Right to Left
	<<= >>=		Right to Left
15	,	Comma	Left to Right

- Q.61(a) What is scope of variable in 'C' meant? Explain type casting in detail also. [R.T.U. 2015]

OR

What is meant by scope of variables in C? What are the different types of variables on the basis of scope? List and explain with example? [Raj. Univ. 2005]

- (b) Explain the difference between automatic and static variable. [R.T.U. 2015]

Ans.(a) In C the scope of variables means that in functions the value of the variable would be available.

C provides a variety of storage class specifiers that can be used to declare explicitly, scope and life time of variables. The storage class decides the portions of the program within which the variables are recognized. There are basically two types of variables on the basis of scope.

- (1) Global Variable
- (2) Local Variables

Example :

```
int m;
main ()
{
```

```

    int i;
    float balance;
    _____
    _____

    function1 ();
}
function1 ()
{
    int i;
    float sum;
    _____
    _____

}

```

1. Global Variables : The variable *m* which has been declared before the main is called global variable. It can be used in all the functions in the program. It need not be declare in other functions. A global variable is also known as an external variable.

2. Local Variables : The variables *i*, *balance* and *sum* are called local variables because they are declared inside a function. Local variables are visible and meaningful only inside the function, in which they are declared. They are not known to other function. The storage class has another qualities (like long or unsigned) that can be added to a variable

```

auto int count;
register char ch;
static int x;
extern long total ;

```

Storage
Auto

Meaning

Local variable which known only to the function in which it is declared. Default is auto.

Static

Local variable which exist and retain it value even after the control transferred to the calling function.

Extern

Global variable known to all functions in the file.

Register

Local variable which is stored in the register.

Type Casting or Conversion : Refer to Q.5.

Ans.(b) Difference between Automatic and Static Variable

S. No.	Static Variable	Automatic Variable
1.	We have to specify the storage class to make a variable static.	It is the default storage class.
2.	If it is not assigned any value then it will give 0 as output.	If it is not assigned any value then it will give garbage value as output.
3.	It is visible to the block in which it is declared and also in the function where it will passed.	It is visible to the block in which the variable is declared.
4.	It retains its value between different function calls. It holds its last value.	It retains its value till the control remains in the block in which the variable is declared.
5.	Static variable should be compile by compiler first.	Auto variable will compile by the compiler after the static variable.

□□□